Comunicador Distribuído de Grupo

Relatório do Trabalho 1 de Sistemas Distribuídos

Campo	Valor
Aluno	Vinicius Alves Pereira
Matrícula	20223002747
Instituição e disciplina	CEFET, Sistemas Distribuídos
Código-fonte e scripts	Github
Demonstração	Demo - Trabalho 1
Aula	Aula - Trabalho 1

Resumo

Este relatório descreve um sistema de bate-papo em grupo inspirado em um grupo de WhatsApp. Há uma única sala, compartilhada por três tipos de cliente que conversam entre si: um cliente desktop em Java (Swing), um cliente web em HTML, CSS e JavaScript e um terceiro cliente em Python (Tkinter). Todos falam com um middleware central em Java, que autentica usuários, recebe cada mensagem, registra no repositório de conversas e a distribui aos participantes conectados.

O texto descreve a arquitetura e a implantação, o banco de contas (PostgreSQL), o protocolo em JSON sobre TCP e WebSocket, os recursos das interfaces (emoticons, seletor de emojis em grade e figurinhas), a persistência cifrada por dispositivo, os repositórios do servidor com suas retenções, os três logs separados, a cifra manual das conversas (com trechos de código e um exemplo calculado passo a passo), as decisões de concorrência, reconexão e tratamento de falhas, e os passos de instalação, execução e demonstração.

Sumário

- 1. Introdução e tecnologias
  - 1.1 Problema e escopo
  - 1.2 Tecnologias e versões
  - 1.3 Papel de cada módulo
- 2. Arquitetura
  - 2.1 Visão geral
  - 2.2 Diagrama de implantação
  - 2.3 Partida e encerramento do servidor
  - 2.4 Módulos do Middleware
  - 2.5 Camadas de um cliente
  - 2.6 Fluxo de mensagens
  - 2.7 Acesso do navegador ao Middleware
- 3. Banco de dados e autenticação
  - 3.1 Banco escolhido e papel dele
  - 3.2 Esquema de dados
  - 3.3 Cadastro de conta
  - 3.4 Autenticação (login)
  - 3.5 Telas e fluxo no Cliente_Desktop
  - 3.6 Web e cliente em Python
  - 3.7 Operações sem autenticação
  - 3.8 Nenhum cliente acessa dados diretamente
- 4. Protocolo
  - 4.1 Formato e delimitação
  - 4.2 Envelope
  - 4.3 Campos cifrados
  - 4.4 Operações
  - 4.5 Códigos de erro
  - 4.6 Identificação do usuário: a sessão
  - 4.7 Limites
- 5. Recursos dos clientes
  - 5.1 Telas e exibição de mensagens
  - 5.2 Envio por botão e pela tecla Enter
  - 5.3 Conversão de emoticons em emojis
  - 5.4 Seletor visual de emojis
  - 5.5 Figurinhas
- 6. Persistência por dispositivo
  - 6.1 Onde fica o histórico em cada cliente
  - 6.2 Formato do registro

    - 6.3 Chave do histórico
    - 6.4 Incorporação sem duplicar
    - 6.5 Persistência na Web
    - 6.6 Separação entre contas no mesmo dispositivo
- 7. Repositórios do servidor, retenção e limpeza
    - 7.1 Os cinco repositórios
    - 7.2 Registro de conversa em JSON
    - 7.3 Retenção de cada repositório
    - 7.4 Quando a limpeza ocorre
    - 7.5 Configuração
    - 7.6 Reescrita atômica e concorrência
- 8. Monitoramento e logs
    - 8.1 Três logs separados
    - 8.2 Log de acessos
    - 8.3 Log de envios
    - 8.4 Log de erros
    - 8.5 Mascaramento de segredos
    - 8.6 Exceções dos clientes enviadas ao servidor
    - 8.7 Cliente sem conexão e proteção contra laço de reenvio
- 9. Criptografia manual
    - 9.1 A chave
    - 9.2 Formato dos bytes cifrados
    - 9.3 Primitivas de 32 bits
    - 9.4 O algoritmo: passos A a D
    - 9.5 Etapas de cifragem e decifragem
    - 9.6 Representação dos bytes
    - 9.7 Código da cifra
    - 9.8 Exemplo completo
    - 9.9 Conferência entre Java, JavaScript e Python
    - 9.10 Distribuição das chaves
    - 9.11 Módulos que conhecem cada chave e onde a mensagem fica legível
    - 9.12 Metadados expostos e justificativa
    - 9.13 Bytes, texto, emojis, figurinhas, imagens e acentos
    - 9.14 Dado atravessando o socket cifrado e repositórios cifrados
    - 9.15 Tratamento de senhas (separado da cifra)
- 10. Concorrência, reconexão e falhas
    - 10.1 Concorrência no Middleware
    - 10.2 Reconexão nos clientes
    - 10.3 Falhas e como o sistema reage
- 11. Instalação e execução
    - 11.1 Pré-requisitos e banco de dados
    - 11.2 Chaves e configuração
    - 11.3 Middleware
    - 11.4 Clientes
- 12. Demonstração
- 13. Conclusão, limitações e referências
    - 13.1 Conclusão
    - 13.2 Limitações
    - 13.3 Código-fonte
    - 13.4 Referências

---

1. Introdução e tecnologias

1.1 Problema e escopo

O trabalho pede um comunicador distribuído de grupo, inspirado em um grupo de WhatsApp, com um middleware central e três clientes interoperáveis. A solução cobre comunicação distribuída, persistência, concorrência, monitoramento e criptografia. Ao longo do texto, Middleware designa o servidor central, e Cliente_Desktop, Cliente_Web e Cliente_Terceiro designam, respectivamente, os clientes em Java (Swing), no navegador e em Python (Tkinter).

1.2 Tecnologias e versões

As versões abaixo foram conferidas nos arquivos de build do projeto (`pom.xml` raiz, `middleware/pom.xml`, `cliente-desktop/pom.xml` e `cliente-python/pyproject.toml`).

Componente	Tecnologia	Versão	Uso
Servidor e Cliente_Desktop	Java	21 (`maven.compiler.release`)	Middleware, módulo comum e interface Swing
Build	Maven	3.x, com `maven-compiler-plugin` 3.13.0, `maven-jar-plugin` 3.5.0 e `maven-dependency-plugin` 3.8.1	Compilação e empacotamento dos três módulos Java

Componente	Tecnologia	Versão	Uso
JSON em Java	Jackson Databind	2.17.2	Montagem e leitura dos quadros do protocolo
WebSocket no servidor	Java-WebSocket	1.5.7	Listener da porta 9001
Acesso ao banco	Driver JDBC do PostgreSQL	42.7.4	Contas de usuário
Banco de dados	PostgreSQL	16	Tabela `contas`
Entrega do Cliente_Web	`HttpServer` do JDK (`com.sun.net.httpserver`)	Incluído no JDK 21	Servidor HTTP estático na porta 8080
Cliente_Desktop	Swing e AWT do JDK	Incluído no JDK 21	Telas de login, cadastro e sala
Cliente_Web	HTML, CSS e JavaScript (módulos ES)	Sem framework e sem dependências	Interface, WebSocket e IndexedDB do navegador
Cliente_Terceiro	Python com Tkinter	3.11 ou superior (`requires-python`), sem dependências externas	Interface gráfica e núcleo do cliente
Cifra das conversas	Implementação própria nas três linguagens	Formato de versão 1	Cifragem e decifragem manuais

O SHA-256 do JDK (`MessageDigest`) é usado somente no tratamento de senhas, que é um mecanismo separado da cifra das conversas (seção 9.15). A cifra das conversas usa apenas aritmética de inteiros de 32 bits e funções de codificação (UTF-8 e Base64). A única função pronta de números aleatórios é usada para sortear o nonce e as chaves.

1.3 Papel de cada módulo

Módulo ou pasta	Conteúdo	Quem usa
`comum/`	Cifra manual, protocolo (`ProtocoloJson`, `LeitorDeLinhas`, `CodigoErro`), conversor de emoticons e mascaramento de segredos	Middleware e Cliente_Desktop
`middleware/`	Listeners TCP, WebSocket e HTTP, processador do protocolo, sessões, contas, repositórios em arquivo, logs e limpeza	Servidor
`cliente-desktop/`	Interface Swing, conexão TCP, histórico local e reporte de erros	Máquina do Desktop
`cliente-web/`	Página, módulos JavaScript, histórico em IndexedDB e reporte de erros	Navegador
`cliente-python/`	Pacote `comunicador`, com `nucleo` (lógica) e `gui` (Tkinter)	Máquina do cliente Python
`recursos/`	`emoticons.json`, `emojis.json`, figurinhas e avatares em PNG, usados pelos três clientes	Os três clientes
`sql/`, `scripts/`, `config/`	Esquema do banco, utilitários de execução e exemplos de configuração	Operação do sistema

---

2. Arquitetura

2.1 Visão geral

O sistema é um bate-papo de sala única. Um processo central, o Middleware, autentica os usuários, recebe cada mensagem, grava nos repositórios do servidor e a distribui a todos os conectados. Três tipos de cliente participam da mesma sala.

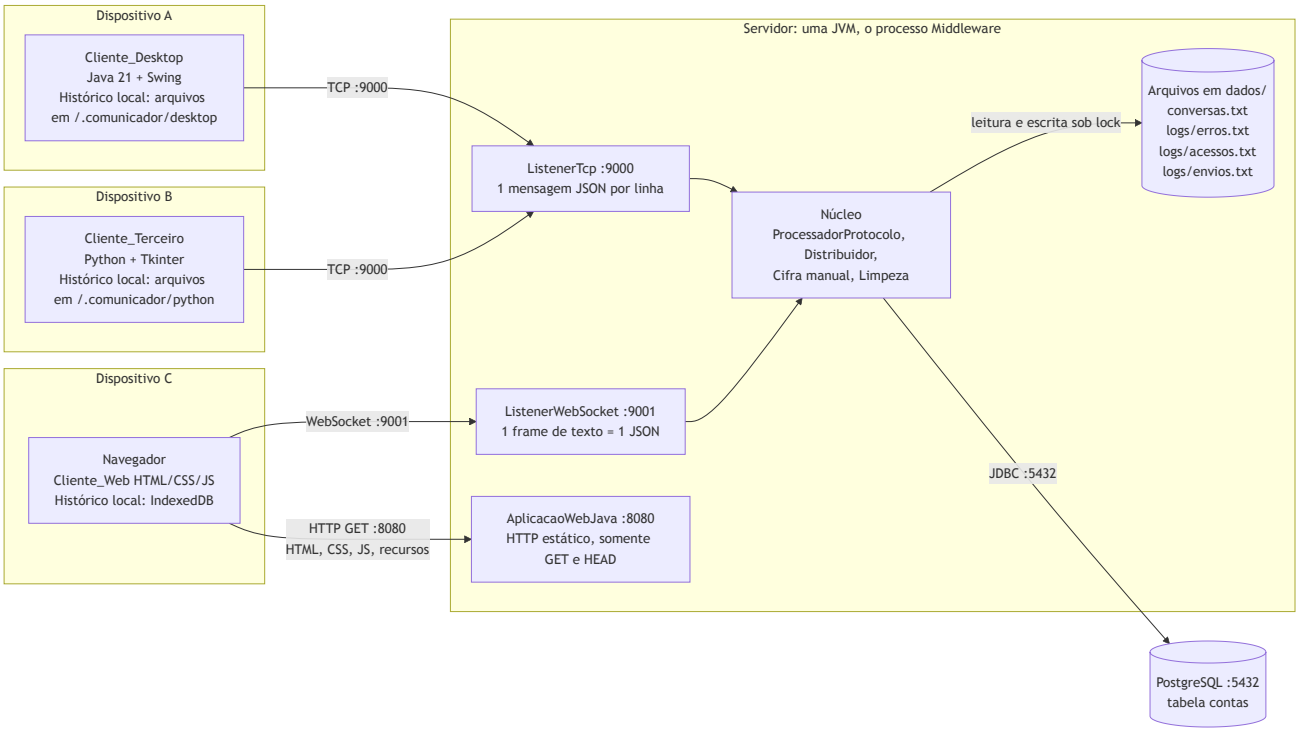
Cliente	Tecnologia	Como chega ao Middleware
Cliente_Desktop	Java 21, Swing	Socket TCP, porta 9000
Cliente_Terceiro	Python 3.11 ou superior, Tkinter	Socket TCP, porta 9000
Cliente_Web	HTML, CSS e JavaScript	WebSocket, porta 9001 (os arquivos da página vêm por HTTP, porta 8080)

A arquitetura se apoia em cinco decisões. A primeira é ter um único processo de servidor: uma JVM executa o listener TCP, o listener WebSocket, o servidor HTTP estático, a lógica do protocolo, os repositórios em arquivo e a limpeza, e o PostgreSQL é o único outro processo do lado do servidor. A segunda é ter um único ponto de acesso aos dados. Somente o Middleware fala com o PostgreSQL, com o arquivo de conversas e com os logs, e os clientes só chegam a esses dados pelo protocolo.

A terceira decisão é usar um único protocolo. TCP e WebSocket carregam o mesmo envelope JSON e as mesmas operações, de modo que os dois listeners são apenas adaptadores de transporte sobre o mesmo código. A quarta é implementar a cifra manual em Java, JavaScript e Python com o mesmo algoritmo, então cada implementação decifra o que a outra cifrou. A quinta é usar três chaves distintas, uma para cada canal protegido: trânsito, repositório do servidor e histórico local.

2.2 Diagrama de implantação

O diagrama mostra máquinas, processos, portas e conexões. Os três dispositivos de cliente são um exemplo. Dois clientes podem rodar na mesma máquina, e o servidor pode dividir a máquina com um cliente.



2.3 Partida e encerramento do servidor

Na partida, a classe `MiddlewareMain` carrega `config/servidor.properties` e `config/chaves.properties`, com variáveis de ambiente sobrepondo as propriedades. Uma configuração inválida (chaves iguais ou mal formadas, retenção ou intervalo menor ou igual a zero, porta fora do intervalo) recusa a partida com código de saída 2, sem imprimir valores de chave. Em seguida, a classe monta os repositórios, os logs, o processador do protocolo, o distribuidor e as operações.

A limpeza é executada de forma síncrona antes de abrir qualquer porta, para que nenhum cliente veja registros vencidos. Só depois o Middleware abre os listeners TCP e WebSocket e o servidor HTTP. Se uma porta estiver ocupada, a partida é recusada (código de saída 1) e o que já havia sido aberto é fechado. Por último, o Middleware agenda a limpeza periódica.

O encerramento (SIGINT ou SIGTERM) fecha os listeners, para a limpeza periódica e fecha as sessões restantes. Cada passo é isolado, então a falha de um não impede os seguintes.

2.4 Módulos do Middleware

Módulo	Responsabilidade	Observação
`ListenerTcp`	Aceita conexões na porta 9000, cria uma thread virtual por conexão, separa linhas com `LeitorDeLinhas` e entrega cada linha ao processador do protocolo	Sem nenhum byte recebido por 90 s, a conexão é encerrada (`SO_TIMEOUT`)
`ListenerWebSocket`	Aceita conexões na porta 9001, confere o cabeçalho `Origin` no handshake, entrega cada frame de texto ao processador e recusa frame binário com `FORMATO_INVALIDO`	Cada conexão tem uma fila de entrada de 256 itens e uma thread virtual que a consome em ordem
`AplicacaoWebJava`	`HttpServer` do JDK com `/` (pasta `cliente-web/`) e `/recursos/` (pasta `recursos/`)	Não conhece chaves, banco, repositórios nem logs
`ProcessadorProtocolo`	Valida o envelope, despacha a operação, aplica limites e trata exceções (`ERRO_INTERNO` e registro no log de erros)	Único ponto de lógica do protocolo e dono do `ordemLock`
`OperacoesAutenticacao`	`register` e `login`: decifra `dados`, valida, verifica a senha, cria a identidade da sessão e registra no log de acessos	Registra a sessão no distribuidor antes de enviar o histórico
`OperacoesConversa`	`send_message`, `history`, `logout` e `error_report`	`send_message` é a seção crítica descrita na seção 2.6
`EnviadorHistorico`	Lê a janela de 7 dias, converte da chave do repositório para a de trânsito e envia em lotes de até 100 mensagens	Usado no login e na operação `history`
`SessaoEnfileirada`	Estado de uma conexão: identidade, fila de saída limitada a 1000 itens e thread escritora própria	`enviar` apenas enfileira
`DistribuidorEmMemoria`	Conjunto das sessões conectadas, com `publicar` que enfileira o quadro em cada uma	`CopyOnWriteArrayList`: muitas leituras e poucas escritas
`ContasRepositorioJdbc`	Acesso ao PostgreSQL com `PreparedStatement` e uma conexão por operação	A violação de unicidade (SQLState `23505`) vira `LoginDuplicadoException`
`TratamentoSenhas`	SHA-256 iterado com salt por conta	Independente da cifra das conversas
`RepositorioArquivo`	Arquivo de texto com um JSON por linha e `ReentrantLock` próprio, com `anexar`, `lerLinhas` e `reescrever` atômico	Base dos quatro arquivos

Módulo	Responsabilidade	Observação
`Limpeza`	Remove registros vencidos de cada arquivo conforme sua retenção	Seção 7

2.5 Camadas de um cliente

Os três clientes têm a mesma divisão lógica, com a lógica separada da interface.

Camada	Responsabilidade	Desktop	Web	Python
Interface	Telas de login, cadastro e sala, seletor de emojis e de figurinhas	`ui/` (Swing)	`index.html`, `main.js`, `css/`	`gui/` (Tkinter)
Protocolo	Montar e interpretar quadros, separar linhas no TCP	`comum` (`ProtocoloJson`, `LeitorDeLinhas`)	`protocolo.js`	`nucleo/protocolo.py`
Cifra	Cifra manual	`comum` (`CifraManual`)	`cifra.js`	`nucleo/cifra.py`
Conexão	Abrir o transporte, enviar, receber, pingar e reconectar	`rede/ConexaoTcp`	`conexao.js` (WebSocket)	`nucleo/conexao.py`
Histórico	Histórico local cifrado e incorporação por `mid`	`historico/HistoricoLocal`	`historico.js` (IndexedDB)	`nucleo/historico.py`
Erros	Reporte de exceções e buffer offline	`erros/`	`erros.js`	`nucleo/erros.py`
Emoticons	Conversão de emoticons	`comum` (`ConversorEmoticons`)	`emoticons.js`	`nucleo/emoticons.py`

2.6 Fluxo de mensagens

Login

O cliente cifra as credenciais com a chave de trânsito. O Middleware decifra em memória, verifica o hash da senha, cria a identidade da sessão e, depois de responder, envia a janela de sete dias do repositório de conversas.

A sessão é registrada no distribuidor antes da leitura do histórico. Uma mensagem publicada entre o registro e a leitura chega duas vezes (ao vivo e no histórico), e o cliente descarta a duplicata pelo `mid`. Na ordem inversa, mensagens poderiam se perder. Perder uma mensagem é pior do que recebê-la duas vezes, por isso a duplicata é o lado escolhido.

Envio e distribuição

O remetente vem da sessão e nunca do conteúdo da requisição. Um campo `remetente` em `send_message` é ignorado, como qualquer campo desconhecido.

As operações de cifra acontecem fora do lock. Cifrar para o repositório e para o trânsito não depende de `mid` nem de `ts`, então isso ocorre antes de tomar o `ordemLock`, o que encurta a seção crítica. O quadro distribuído usa um nonce novo, diferente do usado no repositório.

O remetente só grava a mensagem no histórico local ao receber o eco. Assim, `mid` e `ts` sempre vêm do servidor, e remetente e destinatários seguem o mesmo caminho de código.

Se a gravação no repositório falhar, o Middleware responde `ERRO_INTERNO` e não registra a mensagem no log de envios nem a distribui. Se falhar apenas a gravação no log de envios, o erro vai ao log de erros e a distribuição continua, porque não distribuir uma mensagem já gravada deixaria o repositório e as salas divergentes.

Entrada e saída de dados

A thread que processa a requisição de um remetente nunca escreve no socket de outro cliente. Ela apenas enfileira, e a thread escritora de cada sessão faz a escrita. Esse desacoplamento impede que um cliente lento atrase os demais.

2.7 Acesso do navegador ao Middleware

O navegador abre um WebSocket direto ao Middleware, na porta 9001. Os arquivos estáticos do Cliente_Web (HTML, CSS, JavaScript, imagens e JSON de recursos) são servidos por um servidor HTTP leve embutido na mesma JVM, na porta 8080. A parte de servidor da aplicação web fica, portanto, inteiramente em Java. O navegador não abre um socket TCP Java diretamente, mas suporta WebSocket de forma nativa. Por isso essa escolha dispensa uma aplicação web intermediária, e o Cliente_Web chega ao mesmo processador de protocolo usado pelos clientes TCP.

---

3. Banco de dados e autenticação

3.1 Banco escolhido e papel dele

O banco de dados é o PostgreSQL 16, que é um banco cliente-servidor (processo próprio, porta 5432, acesso por rede e por JDBC). Não é um banco embarcado nem um arquivo local. Ele guarda somente as contas dos usuários. As conversas ficam em arquivo de texto no servidor, como exige o enunciado, e os logs ficam em arquivos separados.

Somente o Middleware se conecta ao banco, e a porta 5432 não precisa (nem deve) ser aberta para a rede. As credenciais de conexão pertencem ao servidor e ficam em `config/servidor.properties` (propriedades `db.url`, `db.usuario` e `db.senha`) ou em variáveis de ambiente, como `DB_SENHA`. Nenhum cliente conhece essas credenciais, e o arquivo real de configuração não é versionado.

O acesso é feito por `ContasRepositorioJdbc`, que usa apenas `PreparedStatement` e abre uma conexão por operação. Cadastro e login são operações raras, então não há pool. Se o banco estiver inacessível, o Middleware continua de pé, e as operações `register` e `login` respondem `SERVICO_INDISPONIVEL`.

3.2 Esquema de dados

O esquema está em `sql/01_schema.sql` e tem uma única tabela.

```
-- Esquema do Banco_de_Dados (PostgreSQL) do chat em grupo distribuido.
--
-- Uso (em um banco vazio):
--   psql -h localhost -U <usuario> -d <banco> -f sql/01_schema.sql
--
-- A unicidade do login e garantida pela restricao UNIQUE: a violacao
-- (SQLState 23505) vira LOGIN_DUPLICADO no Middleware, sem consulta previa
-- e sem condicao de corrida entre cadastros simultaneos.

CREATE TABLE contas (
    id          BIGSERIAL    PRIMARY KEY,
    login       VARCHAR(20)  NOT NULL,
    apelido     VARCHAR(30)  NOT NULL,
    senha_hash  VARCHAR(200) NOT NULL,          -- sha256i$N$salt$hash (nunca a senha legivel)
    avatar      VARCHAR(5)   NOT NULL,          -- av_01 .. av_12
    cor         CHAR(7)      NOT NULL,          -- #RRGGBB maiusculo
    criada_em   TIMESTAMPTZ  NOT NULL DEFAULT now(),
    CONSTRAINT uq_contas_login UNIQUE (login),
    CONSTRAINT ck_login  CHECK (login  ~ '^[a-z0-9_]{3,20}$'),
    CONSTRAINT ck_avatar CHECK (avatar ~ '^av_[0-9]{2}$'),
    CONSTRAINT ck_cor    CHECK (cor    ~ '^#[0-9A-F]{6}$')
);
```

Coluna ou restrição	Significado
`id`	Chave primária sequencial (`BIGSERIAL`)
`login`	Nome de usuário, único. De 3 a 20 caracteres entre letras minúsculas, dígitos e sublinhado
`apelido`	Nome exibido na sala, de 1 a 30 caracteres
`senha_hash`	Resultado do tratamento de senhas no formato `sha256i$N$salt$hash`. A senha legível nunca é gravada (seção 9.15)
`avatar`	Identificador do avatar, de `av_01` a `av_12`. A imagem em si é um PNG em `recursos/avatares/`
`cor`	Cor do apelido na sala, no formato `#RRGGBB` com letras maiúsculas
`criada_em`	Data e hora de criação, preenchida pelo banco
`uq_contas_login`	Garante login único. A violação (SQLState `23505`) é convertida em `LOGIN_DUPLICADO`
`ck_login`, `ck_avatar`, `ck_cor`	O próprio banco recusa valores fora do formato, mesmo que uma validação do Middleware falhe

A unicidade é garantida pelo banco, sem consulta prévia. Assim, dois cadastros simultâneos com o mesmo login não geram condição de corrida: um deles recebe a violação de unicidade e a resposta `LOGIN_DUPLICADO`.

3.3 Cadastro de conta

O cadastro é uma operação do middleware (`register`). O cliente envia login, apelido, senha, avatar e cor dentro do campo `dados`, cifrado com a chave de trânsito (seção 4.3). O Middleware decifra em memória e valida os campos nesta ordem fixa: `login`, `apelido`, `senha`, `avatar` e `cor`. Primeiro verifica a ausência de cada campo e depois o formato.

Campo	Regra
`login`	`^[a-z0-9_]{3,20}$`
`apelido`	De 1 a 30 pontos de código, depois de remover espaços das pontas
`senha`	De 6 a 72 pontos de código. Nunca é gravada, pois vira o valor do tratamento de senhas
`avatar`	De `av_01` a `av_12`
`cor`	`^#[0-9A-Fa-f]{6}$`, guardada em maiúsculas

Se tudo estiver correto, o Middleware gera um salt novo, calcula o hash e insere a linha na tabela `contas`. As contas usadas na demonstração são criadas pela tela de cadastro de qualquer cliente. Não existem contas pré-carregadas.

Erros possíveis: `CAMPO_AUSENTE`, `CAMPO_INVALIDO`, `FORMATO_INVALIDO`, `DECIFRAGEM_FALHOU`, `LOGIN_DUPLICADO` e `SERVICO_INDISPONIVEL`.

3.4 Autenticação (login)

O `login` autentica a conexão e cria a sessão. O cliente envia `{"login","senha"}` cifrado com a chave de trânsito. No Middleware, se a conexão já está autenticada, a resposta é `JA_AUTENTICADO`. Caso contrário, o Middleware valida `dispositivo` e `cliente`, quando presentes, e responde `CAMPO_INVALIDO` se

o formato estiver errado. Depois decifra `dados`, e as falhas desse passo geram `CAMPO_AUSENTE`, `FORMATO_INVALIDO`, `DECIFRAGEM_FALHOU` ou `CAMPO_INVALIDO`.

Em seguida o Middleware verifica a senha. Login inexistente e senha incorreta recebem exatamente a mesma resposta (`CREDENCIAIS_INVALIDAS`), com trabalho de hash equivalente nos dois casos, para que nem a resposta nem o tempo revelem quais logins existem. Na quinta falha de autenticação na mesma conexão, o Middleware responde e encerra a conexão (motivo `falhas_de_login`). Em caso de sucesso, cria a sessão com o perfil da conta, registra o evento `login_ok` no log de acessos, responde com o identificador da sessão e o perfil e, por fim, envia o histórico dos últimos sete dias.

O evento `login_recusado` é gravado no log de acessos a cada credencial inválida. O login tentado só é gravado se casar com o formato de login válido, para não registrar por engano uma senha digitada no campo de login.

3.5 Telas e fluxo no Cliente_Desktop

O Cliente_Desktop inicia em um formulário de login e senha. A raiz da interface é um `CardLayout` (classe `PainelPrincipal`) com duas telas: `TelaLogin` e `TelaSala`. A `TelaSala` não existe até que o método `mostrarSala` seja chamado, e isso só acontece depois da resposta `ok:true` do Middleware ao login. Ao sair, a sala é removida de novo. Na `TelaLogin`, o botão de cadastro abre a tela de cadastro (outro `CardLayout` interno), com seleção de avatar em grade e seleção de cor pelo `JColorChooser`.

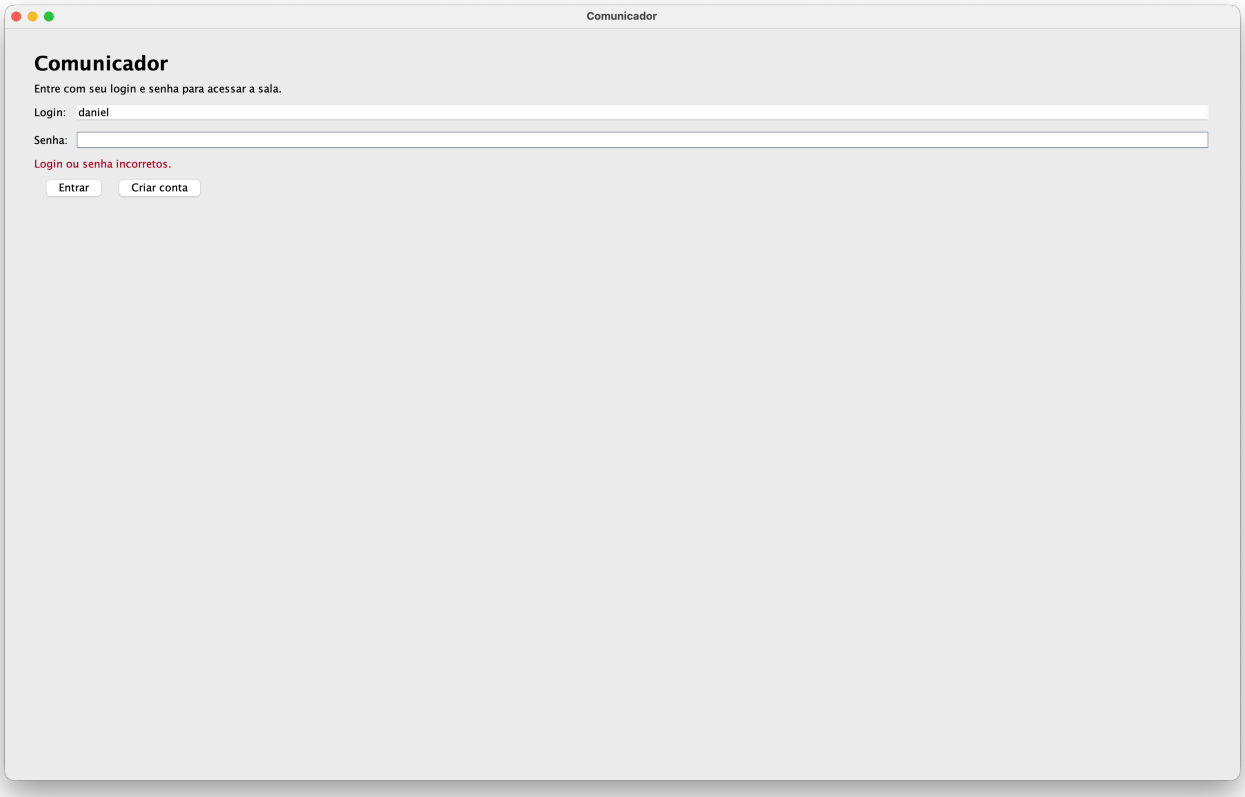


Figura 1: rejeição de credenciais inválidas no Cliente_Desktop.

3.6 Web e cliente em Python

O Cliente_Web e o Cliente_Terceiro autenticam pelo mesmo middleware, com a mesma operação `login` e os mesmos critérios. A diferença está apenas no transporte (WebSocket na porta 9001 para a Web e TCP na porta 9000 para o Python). A rejeição de credenciais inválidas e a de operações sem autenticação são idênticas para os três clientes, porque dependem do processador do protocolo, que não conhece o transporte.

3.7 Operações sem autenticação

As operações `send_message`, `history` e `logout` exigem uma sessão autenticada. Em uma conexão sem sessão, o Middleware responde `NAO_AUTENTICADO` sem executar nenhuma lógica da operação. A verificação ocorre antes do limite de requisições, então uma operação sem sessão nem chega a consumir limite.

3.8 Nenhum cliente acessa dados diretamente

Os clientes não têm driver JDBC, nem caminho para `dados/`, nem credenciais do banco. Toda consulta passa pelo protocolo. No servidor HTTP estático, a pasta `dados/` nunca é raiz de contexto, e tentativas de sair da raiz com `..` ou percent-encoding recebem resposta 403.

---

4. Protocolo

4.1 Formato e delimitação

Cada mensagem do protocolo é um objeto JSON codificado em UTF-8. Não trafega nenhum objeto Java serializado. Java, JavaScript e Python interpretam os mesmos bytes, o que torna o protocolo independente de linguagem.

Transporte	Porta padrão	Delimitador	Observações
TCP (Desktop e Python)	9000 (`porta.tcp`)	`\n` (LF, byte `0x0A`) ao final de cada mensagem, ou seja, uma mensagem JSON por linha	O receptor remove um `\r` final, se houver
WebSocket (Web)	9001 (`porta.ws`)	Um frame de texto equivale a uma mensagem JSON, sem `\n` obrigatório	Um `\n` final é tolerado. Frame binário é recusado com `FORMATO_INVALIDO` e a conexão continua aberta

O delimitador `\n` é inequívoco porque o emissor usa JSON compacto, sem espaços e sem quebras de linha, e o JSON escapa quebras de linha dentro de strings como `\n`. Uma linha serializada, portanto, nunca contém um LF literal. Caracteres não ASCII (acentos e emojis) saem como UTF-8 literal, e a entrada também aceita escapes `\uXXXX`, inclusive pares substitutos.

Como o receptor TCP separa as linhas

O `LeitorDeLinhas` (Java) e o equivalente em Python recebem bytes em pedaços de qualquer tamanho, inclusive um byte por vez, e devolvem as linhas completas. O resultado não depende de como o fluxo foi partido. Uma linha só é entregue quando o `\n` chega, e bytes sem `\n` ficam pendentes. O `\r` imediatamente antes do `\n` é removido. Uma linha vazia é entregue com conteúdo vazio e, ao ser interpretada, vira `FORMATO_INVALIDO`.

Cada linha tem o limite de 1 MiB (1.048.576 bytes). Quando uma linha passa do limite, o leitor emite uma única linha marcada como grande demais, descarta os bytes até o próximo `\n` e segue normalmente. O Middleware responde `MENSAGEM_MUITO_GRANDE` e mantém a conexão aberta. Por fim, bytes que não formam UTF-8 válido geram `FORMATO_INVALIDO`, pois a decodificação é estrita.

No WebSocket o mesmo limite é conferido pelo processador sobre o tamanho em UTF-8 do frame.

4.2 Envelope

Requisição (cliente para Middleware)

Campo	Tipo	Obrigatório	Descrição
`v`	inteiro	sim	Versão do protocolo. O valor atual é `1`. Outro inteiro gera `VERSAO_NAO_SUPORTADA`
`op`	texto	sim	`register`, `login`, `send_message`, `history`, `error_report`, `ping` ou `logout`. Diferencia maiúsculas de minúsculas
`id`	texto de 1 a 64 caracteres	não	Identificador de correlação escolhido pelo cliente. O Middleware o devolve na resposta

Campos desconhecidos são ignorados, o que permite evoluir o protocolo. Isso inclui um eventual campo `remetente` em `send_message` (seção 4.6).

Resposta (Middleware para cliente)

A resposta repete `v`, `op` e `id` da requisição e acrescenta `ok`. Se a requisição não tinha `id`, a resposta traz `"id":null`.

```
{"v":1,"op":"login","id":"l1","ok":true,"sessao":"3f9a1c0de2b44f6a8c7d5e4b3a291807","perfil":
        {"login":"ana","apelido":"Ana","avatar":"av_03","cor":"#1E88E5"}}
{"v":1,"op":"login","id":"l1","ok":false,"erro":{"codigo":"CREDENCIAIS_INVALIDAS","mensagem":"Login ou senha invalidos"}}
```

Em respostas de erro, `erro.codigo` é o código (seção 4.5), `erro.mensagem` é um texto fixo por código que nunca repete dados recebidos, e `erro.campo` aparece somente quando o erro aponta um campo.

Quando o envelope é inválido (JSON inválido, `v` ou `op` ausentes, tipos errados), a versão não é suportada ou a operação é desconhecida, a resposta usa `"op":"erro"` e `"id":null`:

```
{"v":1,"op":"erro","id":null,"ok":false,"erro":{"codigo":"VERSAO_NAO_SUPORTADA","mensagem":"Versao do protocolo nao
        suportada","campo":"v"}}
```

Eventos enviados pelo Middleware sem requisição

`op`	Quando	Campos
`message`	Uma mensagem é aceita. Vai a todas as sessões registradas, incluindo a do remetente (eco)	`v`, `op`, `mensagem`. Sem `id` e sem `ok`
`history`	Logo após um `login` com sucesso (histórico automático)	Mesmo formato da resposta a `history`, com `"id":null` e `"ok":true`

Objeto `mensagem`

É usado no evento `message` e em cada item de `mensagens` do `history`.

Campo	Tipo	Descrição
`mid`	texto (UUID v4)	Identificador da mensagem, atribuído pelo Middleware
`remetente`	objeto	Perfil público: `login`, `apelido`, `avatar` e `cor`
`ts`	texto	Data e hora do servidor, ISO-8601 em UTC com milissegundos e `Z`, por exemplo `2025-03-14T18:22:05.431Z`
`tipo`	texto	`texto` ou `figurinha`
`conteudo`	texto	Base64 da cifra manual com a chave de trânsito. Para `texto`, cifra o texto. Para `figurinha`, cifra o identificador `fig_NN`

O `ts` é sempre atribuído pelo Middleware, com o relógio do servidor truncado para milissegundos.

4.3 Campos cifrados

Dois campos carregam dados cifrados com a cifra manual e a chave de trânsito, sempre em Base64 padrão com preenchimento `=`.

Campo	Onde	Texto claro antes de cifrar
`dados`	Requisições `register` e `login`	JSON compacto com os campos da operação
`conteudo`	Requisição `send_message`, evento `message` e itens de `history`	O texto da mensagem ou o identificador da figurinha

A camada de protocolo não conhece chaves nem a cifra, e trata esses campos como textos opacos. Quem decifra no Middleware é o manipulador da operação. Cada mensagem distribuída é recifrada pelo Middleware com um nonce novo, então o `conteudo` do evento `message` é diferente do `conteudo` enviado pelo remetente. O Middleware gera um único quadro por mensagem, e todas as sessões recebem exatamente o mesmo `conteudo`.

Falha de decifragem (MAC inválido, versão desconhecida, Base64 inválido ou UTF-8 inválido) gera `DECIFRAGEM_FALHOU`. O log de erros recebe um texto fixo, sem o conteúdo recebido.

4.4 Operações

Operação	Exige sessão autenticada	Limite por conexão	Resposta de sucesso (além de `v`, `op`, `id` e `ok:true`)
`register`	não	nenhum	`login`
`login`	não	nenhum	`sessao` e `perfil`, seguidos do histórico automático
`send_message`	sim	30 por 10 s	`mid` e `ts`
`history`	sim	nenhum	Um ou mais quadros `history`
`error_report`	não	10 por minuto	`aceitos`
`ping`	não	nenhum	`ts`
`logout`	sim	nenhum	Nenhum campo extra, e a conexão é encerrada

A ordem de verificação no processador é: tamanho (1 MiB), envelope (`FORMATO_INVALIDO` e `VERSAO_NAO_SUPORTADA`), operação conhecida (`OPERACAO_DESCONHECIDA`), tamanho do `id`, autenticação (`NAO_AUTENTICADO`), limite de requisições (`LIMITE_EXCEDIDO`) e, por fim, a lógica da operação. Os limites são contados por conexão e por operação, em janela deslizante.

Os valores cifrados nos exemplos desta seção foram calculados com a implementação real da cifra, com a chave fictícia de trânsito `000102...1f` e nonces escolhidos para o exemplo. Em uso real o nonce é sorteado a cada mensagem.

register

Cria uma conta. Não autentica a conexão.

```
{"v":1,"op":"register","id":"r1","dados":"AQECAwQFBgcI9t4o9KDM8EkfiA+BzZyAukK/SqMCC/v5kmsFtXqIv15346GItLLfbj7OVPOnoCHuw4jxytOp0ncPCPOQ/66
```

O texto claro de `dados`, antes de cifrar:

```
{"login":"ana","apelido":"Ana","senha":"senha-de-teste","avatar":"av_03","cor":"#1E88E5"}
```

Resposta: `{"v":1,"op":"register","id":"r1","ok":true,"login":"ana"}`. As regras de cada campo estão na seção 3.3.

login

```
{"v":1,"op":"login","id":"l1","dados":"AQECAwQFBgcI9t4o9KDM8EkfiA+BzZyAulCqQacKTa752ywqs3qH9xkp8qqTofXHMSUnX30ZJwGf","dispositivo":"b1c2d3
        0000-4000-8000-000000000001","cliente":{"tipo":"web","versao":"1.0"}}
```

O texto claro de `dados` é `{"login":"ana","senha":"senha-de-teste"}`.

Campo	Descrição
`dados`	Cifra (trânsito) do JSON com `login` e `senha`
`dispositivo`	Opcional. UUID do dispositivo, `^[0-9A-Za-z-]{1,64}$`. Usado nos logs
`cliente`	Opcional. Objeto com `tipo` (`desktop`, `web` ou `python`) e `versao`

A resposta de sucesso traz `sessao` (identificador de 32 hexadecimais usado apenas para correlacionar logs, não é uma credencial) e `perfil`. Em seguida o Middleware envia o histórico automático.

send_message

```
{"v":1,"op":"send_message","id":"m1","tipo":"texto","conteudo":"AQECAwQFBgcIwpAlt+fC7B5VxU+6FdwkTsY9hw=="}
{"v":1,"op":"send_message","id":"m3","tipo":"figurinha","conteudo":"AQECAwQFBgcI65UjxPeWL/NVQSjq9TY="}
```

Os textos claros são `Ola, grupo!` e `fig_03`. No Middleware, a operação valida primeiro `tipo` (`texto` ou `figurinha`) e `conteudo` (presente e do tipo texto). Depois decifra `conteudo` com a chave de trânsito, apenas em variável local, e valida o texto claro: para `texto`, não vazio e com até 2000 pontos de código; para `figurinha`, no padrão `^fig_[0-9]{2}$`. Se essa validação falhar, a resposta é `CONTEUDO_INVALIDO`.

Com o conteúdo válido, o Middleware toma o `ordemLock` e, dentro dele, atribui `mid` (UUID v4) e `ts`, grava no repositório de conversas (com o conteúdo cifrado pela chave do repositório), grava no log de envios somente `ts` e `login` e publica o evento `message` a todas as sessões. Por fim, responde ao remetente. Se a gravação no repositório falhar, nada é distribuído e a resposta é `ERRO_INTERNO`.

```
{"v":1,"op":"send_message","id":"m1","ok":true,"mid":"6c1f0f0e-8f6b-4f0a-9d8f-3a1b7f7a9c11","ts":"2025-03-14T18:22:05.431Z"}
```

O evento `message` correspondente, enviado a todos, repete o mesmo texto claro com outro nonce e portanto outro Base64:

```
{"v":1,"op":"message","mensagem":{"mid":"6c1f0f0e-8f6b-4f0a-9d8f-3a1b7f7a9c11","remetente":
        {"login":"ana","apelido":"Ana","avatar":"av_03","cor":"#1E88E5"},"ts":"2025-03-
        14T18:22:05.431Z","tipo":"texto","conteudo":"ARESExQVFhcY1MulN2C+eZqY8Rz+S9jpopU5Eg=="}}
{"v":1,"op":"message","mensagem":{"mid":"0b7d2a54-3c1e-4b8a-9f10-5d6e7f809a1b","remetente":{"login":"bia","apelido":"Bia
        ✨","avatar":"av_07","cor":"#D81B60"},"ts":"2025-03-
        14T18:22:09.002Z","tipo":"figurinha","conteudo":"ARESExQVFhcY/c6jRHDqQEDCVNJhC4s="}}
```

O evento é enfileirado antes da resposta, então o remetente normalmente recebe o eco antes do `ok:true`. Os clientes não dependem dessa ordem e gravam a mensagem no histórico local ao receber o evento, usando o `mid`. O apelido `Bia ✨` mostra que caracteres não ASCII dos perfis viajam como UTF-8 literal.

history

```
{"v":1,"op":"history","id":"h1"}
{"v":1,"op":"history","id":"h2","desde":"2025-03-14T00:00:00.000Z"}
```

O campo `desde` é opcional (ISO-8601 em UTC). O início é limitado à janela de sete dias, então um `desde` mais antigo que `agora - 7 dias` é substituído por esse limite. As mensagens vêm na ordem de gravação, em lotes de até 100:

```
{"v":1,"op":"history","id":"h1","ok":true,"lote":1,"ultimo":false,"mensagens":[ ... 100 mensagens ... ]}
{"v":1,"op":"history","id":"h1","ok":true,"lote":2,"ultimo":true,"mensagens":[ ... ]}
```

O campo `ultimo:true` marca somente o último lote. Sem mensagens na janela, o Middleware envia um único quadro com `lote:1`, `ultimo:true` e `mensagens:[]`. Os lotes só saem depois de montados, então uma falha na leitura nunca produz histórico parcial. Registros do repositório que não decifram são ignorados e relatados uma vez no log de erros, sem conteúdo.

O histórico automático após o login tem o mesmo formato, com `"id":null`:

```
{"v":1,"op":"history","id":null,"ok":true,"lote":1,"ultimo":true,"mensagens":[{"mid":"6c1f0f0e-8f6b-4f0a-9d8f-
        3a1b7f7a9c11","remetente":{"login":"ana","apelido":"Ana","avatar":"av_03","cor":"#1E88E5"},"ts":"2025-03-
        14T18:22:05.431Z","tipo":"texto","conteudo":"ARESExQVFhcY1MulN2C+eZqY8Rz+S9jpopU5Eg=="},{"mid":"0b7d2a54-3c1e-4b8a-9f10-
        5d6e7f809a1b","remetente":{"login":"bia","apelido":"Bia ✨","avatar":"av_07","cor":"#D81B60"},"ts":"2025-03-
        14T18:22:09.002Z","tipo":"figurinha","conteudo":"ARESExQVFhcY/c6jRHDqQEDCVNJhC4s="}]}
```

O perfil do remetente vem da conta no momento do envio do histórico. Se a conta não existe mais, o Middleware usa um perfil neutro com o próprio login, para que a mensagem continue exibível.

error_report

Envia eventos de erro de um cliente. Não exige sessão, porque falhas acontecem também na tela de login e o buffer offline precisa esvaziar mesmo assim.

```
{"v":1,"op":"error_report","id":"e1","eventos":[{"evid":"0e9d1c2b3a4f","ts":"2025-03-
        14T18:30:00.100Z","origem":"cliente","modulo":"conexao.js","tipo":"TypeError","descricao":"falha ao
        enviar","pilha":"TypeError: x\n  at conexao.js:10","dispositivo":"b1c2d3e4-0000-4000-8000-
        000000000001","cliente":"web","tentativas":0}]}
```

O campo `eventos` é obrigatório e deve ser uma lista não vazia de 1 a 20 eventos, e a requisição inteira deve ter até 64 KiB. Acima disso, a resposta é `LIMITE_EXCEDIDO`, o mesmo código usado para o limite de 10 requisições por minuto por conexão. Cada evento precisa de `evid` (`^[0-9A-Za-z._:-]{1,64}$`), `ts` (ISO-8601), `modulo`, `tipo` e `descricao`, e os campos `pilha` e `descartados` são opcionais. Um evento inválido é ignorado e não é confirmado, então o cliente o mantém no buffer offline.

Os eventos válidos têm `descricao` e `pilha` mascaradas (seção 8.5) e vão para o log de erros, com a origem sempre gravada como `cliente`. Um `evid` já gravado recentemente (cache dos últimos 10.000) não é gravado de novo, mas é confirmado em `aceitos`. Assim, um reenvio por perda de confirmação não duplica o log.

Resposta: `{"v":1,"op":"error_report","id":"e1","ok":true,"aceitos":["0e9d1c2b3a4f"]}`. O cliente remove do buffer somente os `evid` presentes em `aceitos`.

ping e logout

`{"v":1,"op":"ping","id":"p1"}` recebe `{"v":1,"op":"ping","id":"p1","ok":true,"ts":"2025-03-14T18:22:05.431Z"}`. Não exige sessão e serve de keep-alive. O Middleware fecha conexões TCP sem nenhum byte recebido por 90 s, e os clientes enviam `ping` a cada 30 s.

`{"v":1,"op":"logout","id":"o1"}` recebe `{"v":1,"op":"logout","id":"o1","ok":true}`. O Middleware deixa de distribuir novos quadros à sessão, esvazia a fila de saída e encerra a conexão com o motivo `logout`.

4.5 Códigos de erro

São 15 códigos, definidos em `CodigoErro.java`. O texto de `erro.mensagem` é fixo e idêntico nas três implementações. Os textos não têm acentos de propósito.

Código	`erro.mensagem`	Quando ocorre
`FORMATO_INVALIDO`	`Formato de mensagem invalido`	JSON inválido, raiz que não é objeto, `v` ou `op` ausentes ou com tipo errado, `id` fora do formato, bytes que não são UTF-8, frame binário no WebSocket, linha vazia
`MENSAGEM_MUITO_GRANDE`	`Mensagem acima do tamanho maximo`	Linha ou frame acima de 1 MiB
`VERSAO_NAO_SUPORTADA`	`Versao do protocolo nao suportada`	`v` inteiro e diferente de 1
`OPERACAO_DESCONHECIDA`	`Operacao desconhecida`	`op` fora das sete operações
`NAO_AUTENTICADO`	`Autenticacao necessaria`	`send_message`, `history` ou `logout` em conexão sem sessão
`JA_AUTENTICADO`	`Sessao ja autenticada`	`login` em conexão já autenticada
`CREDENCIAIS_INVALIDAS`	`Login ou senha invalidos`	Login inexistente ou senha incorreta (mesma resposta)
`LOGIN_DUPLICADO`	`Login ja cadastrado`	`register` com login existente
`CAMPO_AUSENTE`	`Campo obrigatorio ausente`	Campo obrigatório omitido, nulo ou vazio
`CAMPO_INVALIDO`	`Campo com valor invalido`	Valor fora do formato esperado
`DECIFRAGEM_FALHOU`	`Nao foi possivel decifrar os dados`	MAC inválido, versão da cifra desconhecida, Base64 ou UTF-8 inválido
`CONTEUDO_INVALIDO`	`Conteudo invalido`	Texto vazio, acima de 2000 pontos de código ou identificador de figurinha fora do padrão
`LIMITE_EXCEDIDO`	`Limite de requisicoes excedido`	Mais de 30 `send_message` em 10 s, mais de 10 `error_report` em 1 min, ou `error_report` grande demais
`SERVICO_INDISPONIVEL`	`Servico indisponivel`	Banco de dados inacessível em `register`, `login` ou na leitura de perfis do `history`
`ERRO_INTERNO`	`Erro interno`	Exceção inesperada, com detalhes apenas no log de erros

A conexão permanece aberta em todos os casos, exceto: a quinta falha de autenticação (`falhas_de_login`), o `logout`, a fila de saída da sessão cheia (`fila_cheia`, 1000 itens), a fila de entrada da conexão WebSocket cheia (`fila_cheia`, 256 itens) e 90 s sem dados no TCP (`timeout`).

Exemplos de respostas de erro:

```
{"v":1,"op":"erro","id":null,"ok":false,"erro":{"codigo":"FORMATO_INVALIDO","mensagem":"Formato de mensagem invalido"}}
{"v":1,"op":"register","id":"r2","ok":false,"erro":{"codigo":"CAMPO_AUSENTE","mensagem":"Campo obrigatorio ausente","campo":"apelido"}}
{"v":1,"op":"register","id":"r3","ok":false,"erro":{"codigo":"CAMPO_INVALIDO","mensagem":"Campo com valor invalido","campo":"cor"}}
{"v":1,"op":"send_message","id":"m5","ok":false,"erro":{"codigo":"LIMITE_EXCEDIDO","mensagem":"Limite de requisicoes excedido"}}
```

4.6 Identificação do usuário: a sessão

O remetente de uma mensagem nunca vem do conteúdo da requisição. Depois do `login`, a identidade (login, apelido, avatar, cor, dispositivo e tipo de cliente) fica vinculada à própria conexão, na sessão do Middleware. Em `send_message`, o Middleware usa o perfil dessa sessão. Por isso, um campo `remetente` enviado em `send_message` é tratado como qualquer campo desconhecido e ignorado, e o quadro continua válido.

O campo `sessao` devolvido pelo `login` é um identificador de 16 bytes aleatórios em hexadecimal (32 caracteres). Ele serve apenas para correlacionar registros nos logs. Não é uma credencial, e enviá-lo não autentica nada. Antes do `login`, a sessão não tem identificador, e os registros de log trazem `sessao:null`. O mesmo login pode ter várias sessões simultâneas (vários dispositivos), cada uma em sua conexão.

4.7 Limites

Limite	Valor	Resultado ao exceder
Tamanho de uma mensagem (linha TCP ou frame WebSocket)	1 MiB = 1.048.576 bytes	`MENSAGEM_MUITO_GRANDE`, com conexão mantida
Texto de uma mensagem	2000 pontos de código, após decifrar	`CONTEUDO_INVALIDO`
`id` de correlação	1 a 64 caracteres	`FORMATO_INVALIDO`
`send_message`	30 por 10 s, por conexão	`LIMITE_EXCEDIDO`
`error_report` (frequência)	10 por minuto, por conexão	`LIMITE_EXCEDIDO`

Limite	Valor	Resultado ao exceder
`error_report` (conteúdo)	20 eventos e 64 KiB por requisição	`LIMITE_EXCEDIDO`
Lote de `history`	100 mensagens por quadro	Divide em mais lotes
Alcance de `history`	Janela de 7 dias	`desde` anterior é substituído pelo limite
Falhas de autenticação	5 por conexão	A quinta encerra a conexão
Silêncio no TCP	90 s sem nenhum byte	Conexão encerrada (`timeout`)
Fila de saída por sessão	1000 itens	Sessão encerrada (`fila_cheia`)
Fila de entrada por conexão WebSocket	256 itens	Conexão encerrada (`fila_cheia`)

No WebSocket, o cabeçalho `Origin` do handshake, quando presente, precisa constar em `ws.origens_permitidas`. Caso contrário o handshake é recusado e nenhuma sessão é criada.

---

5. Recursos dos clientes

5.1 Telas e exibição de mensagens

Os três clientes têm três telas: login, cadastro e sala. A sala só aparece depois que o Middleware confirma o login. Em todos, cada mensagem da sala mostra o apelido do remetente na cor escolhida na conta, o avatar, o conteúdo (texto, emojis ou figurinha) e a data e hora no fuso local do dispositivo.

Aspecto	Cliente_Desktop (Swing)	Cliente_Web	Cliente_Terceiro (Tkinter)
Tela de login	`TelaLogin`, com botão de cadastro	Formulário de login, com campo para a chave de trânsito	Tela de login, com botão "Criar conta"
Tela de cadastro	Login, apelido, senha com confirmação, avatar em grade e `JColorChooser`	Login, apelido, senha, avatar em grade e `input type="color"`	Login, apelido, senha, avatar em grade e cor em `#RRGGBB` com seletor do sistema
Sala	`TelaSala`, com painel de bolhas de mensagem	Página com lista de mensagens e formulário de envio	Janela com um `tk.Text` somente leitura, campo de texto e botões
Avatar na sala	40 px	40 px	Até 48 px
Figurinha na sala	120 px	Imagem da figurinha	Até 80 px
Chave de trânsito	Arquivo `<raiz>/.comunicador/chave_transito.txt`	Digitada na tela de login e guardada no `localStorage`	Arquivo `<raiz>/.comunicador/chave_transito.txt`
Data e hora	Fuso local	Fuso local	Fuso local, formato `dd/mm/aaaa HH:MM:SS`

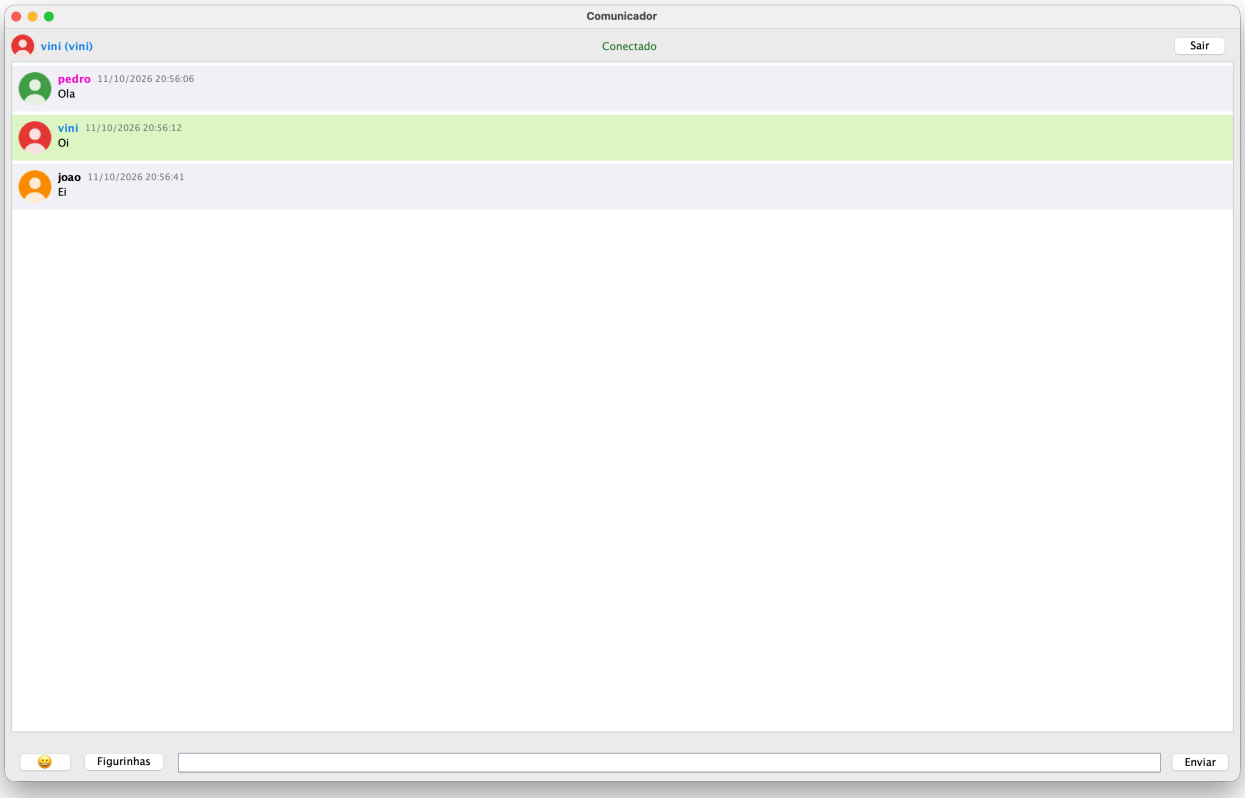


Figura 2: sala de bate-papo do Cliente_Desktop.

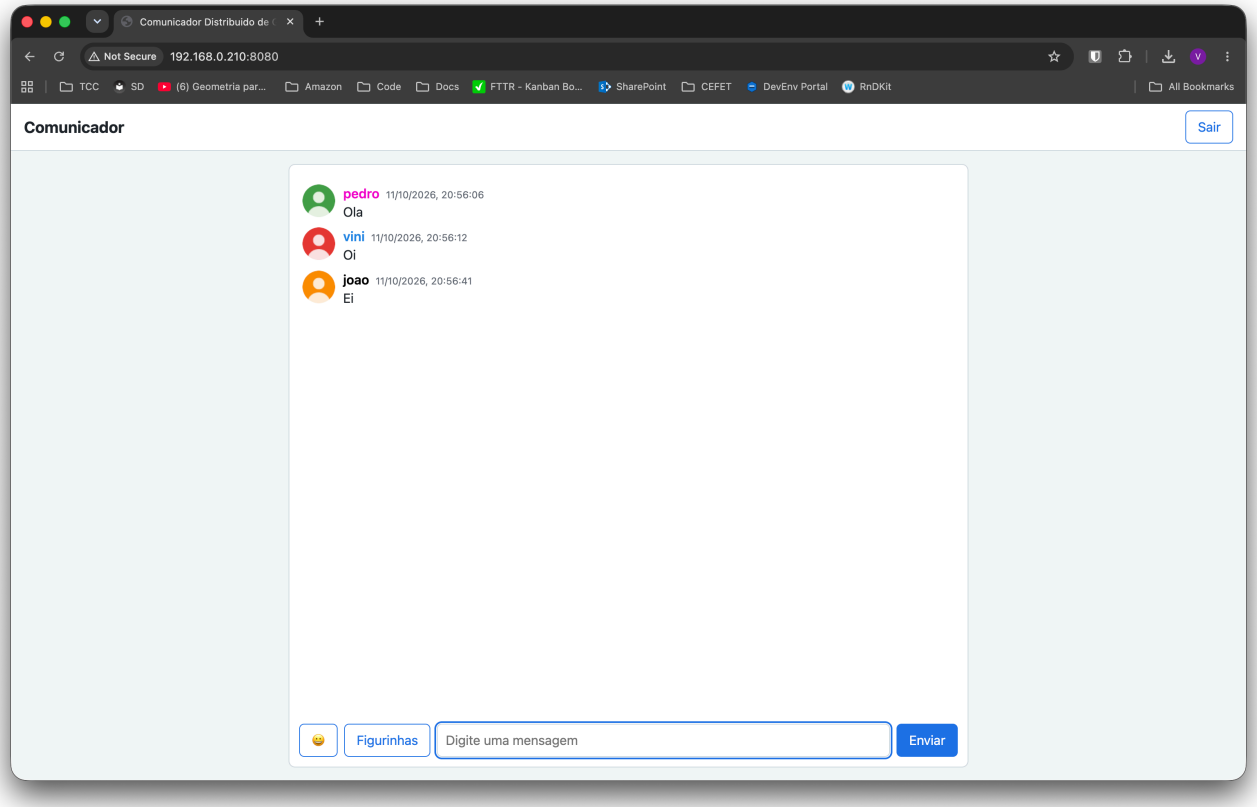


Figura 3: sala de bate-papo do Cliente_Web.

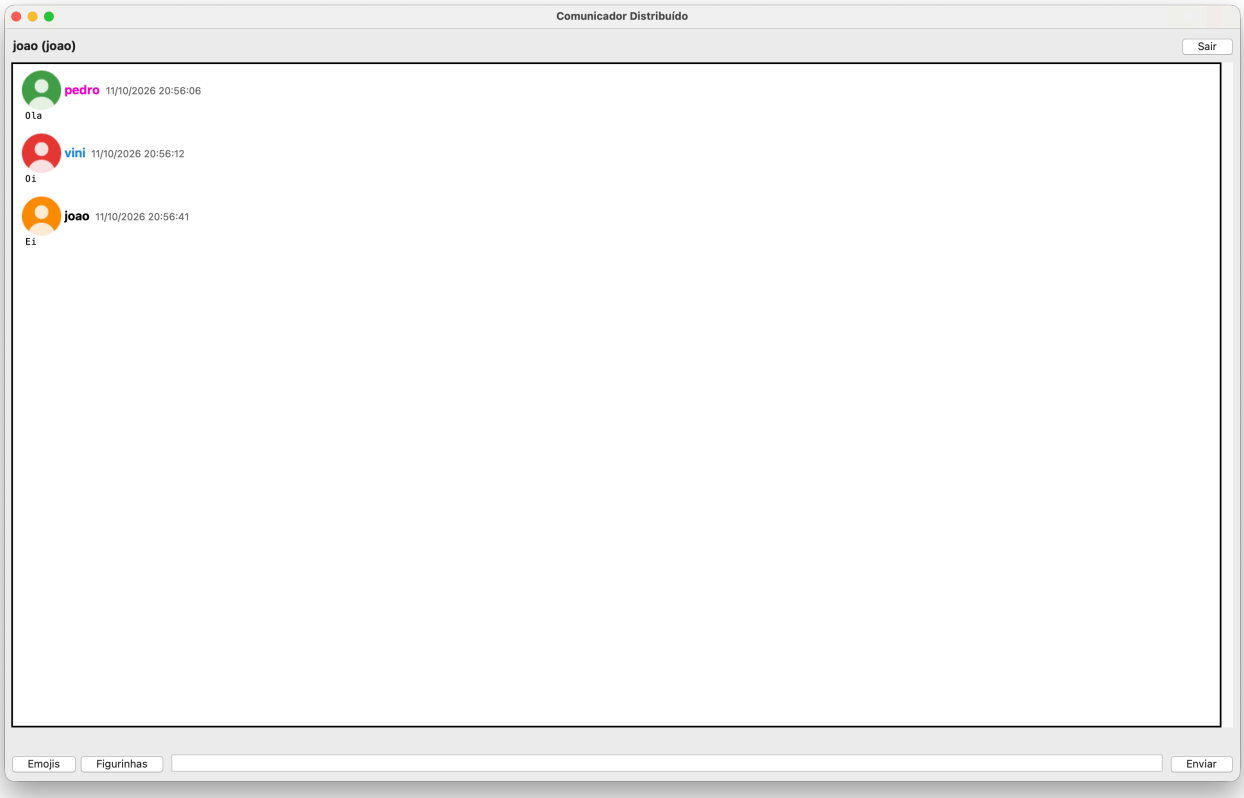


Figura 4: sala de bate-papo do Cliente_Terceiro.

5.2 Envio por botão e pela tecla Enter

Nos três clientes, o botão Enviar e a tecla Enter chamam a mesma função de envio. No Desktop, o campo de texto (`JTextField`) registra um `ActionListener` para o Enter, e o botão Enviar registra outro, que chama o mesmo método `enviar()`. Na Web, o formulário de envio dispara o evento `submit` ao clicar no botão, o campo de texto trata a tecla Enter com o evento `keydown`, e os dois chamam o mesmo controlador de envio (`envio.js`). No Python, o botão e a tecla Enter do `ttk.Entry` chamam a mesma função.

O texto em branco (vazio ou só com espaços Unicode) não é enviado e não altera o campo. O limite de 2000 pontos de código é medido depois da conversão de emoticons. No Middleware, o mesmo limite é conferido de novo ao decifrar o conteúdo.

5.3 Conversão de emoticons em emojis

O usuário pode digitar emoticons como `:-)`. No momento do envio, o cliente converte os emoticons do texto em emojis, e o texto convertido é o que é cifrado e enviado. Quem recebe a mensagem não converte nada, porque o emoji já vem no conteúdo. O Middleware também não converte.

A tabela de conversão está em `recursos/emoticons.json` e é a fonte única dos três clientes. Ela tem 32 pares, agrupados abaixo por emoji:

```
🙂   :-)   :)   =)
😄   :-D   :D
😉   ;-)   ;)
😞   :-(   :(
❤️   <3
😎   B-)   8-)
😐   :-|   :|
😊   ^_^
👍   (y)
😢   :'(
😛   :-P   :P   :p
😮   :-O   :O   :o
😕   :-/   :/
😘   :-*   :*
😠   >:(
😇   O:)
😑   -_-
😆   xD   XD
```

O algoritmo é o mesmo nos três clientes (`ConversorEmoticons.java` em `comum`, `emoticons.js` e `emoticons.py`). O conversor percorre o texto da esquerda para a direita, com um índice `i`. Em cada posição, se a fronteira esquerda é válida, ele tenta os emoticons da tabela do mais longo para o mais curto. Um emoticon corresponde quando o texto o contém a partir de `i` e a fronteira direita é válida. Se corresponde, o conversor emite o emoji e avança o tamanho do emoticon. Caso contrário, emite o caractere atual e avança um.

A fronteira esquerda é válida quando `i` é o início do texto ou o caractere anterior é espaço em branco. A fronteira direita é válida quando o emoticon termina no fim do texto ou o próximo caractere é espaço em branco ou um destes sinais: ponto, vírgula, ponto e vírgula, exclamação ou interrogação. Note a assimetria. A pontuação vale como fronteira depois do emoticon, mas não antes. A regra de fronteira evita converter `:/` dentro de `http://x.com` e `8)` dentro de "item 8) feito".

Exemplos obtidos aplicando o conversor real (`emoticons.py`) à tabela `recursos/emoticons.json`:

Entrada	Saída	Observação
`oi :) tudo bem? :D`	`oi 🙂 tudo bem? 😄`	Conversão básica
`http://x.com :)`	`http://x.com 🙂`	A URL não é alterada
`veja http://x.com/a :/ ok`	`veja http://x.com/a 😕 ok`	O `:/` isolado converte, o da URL não
`item 8) feito`	`item 8) feito`	`8)` não é emoticon da tabela e não tem fronteira esquerda
`:-) :) =)`	`🙂 🙂 🙂`	Variantes do sorriso
`O:) B-) 8-)`	`😇 😎 😎`	Anjo e óculos
`<3 ^_^ -_- (y)`	`❤ 😊 😑 👍`	Coração e outros (o coração é U+2764 seguido de U+FE0F)
`:'( >:(`	`😢 😠`	Lágrima e raiva
`xD XD xd`	`😆 😆 xd`	Diferencia maiúsculas de minúsculas
`:).`	`🙂.`	Ponto depois do emoticon é fronteira válida
`.:)`	`.:)`	Pontuação antes do emoticon não vale
`:):)`	`:):)`	Emoticons colados não convertem
`a:)b`	`a:)b`	Emoticon colado em letras não converte
`:-)))`	`:-)))`	Parênteses a mais impedem a conversão
`(y)(y) (y)`	`(y)(y) 👍`	Só o último tem fronteiras válidas
`olá 😊 :) cão`	`olá 😊 🙂 cão`	Acentos e emoji já presente são preservados

Idempotência. Converter duas vezes dá o mesmo resultado que converter uma vez. Os emoticons da tabela são ASCII e nenhum emoji da tabela contém caractere ASCII, então a segunda passagem não encontra emoticon novo. Todos os exemplos acima foram reconvertidos na execução e não mudaram.

5.4 Seletor visual de emojis

Além da conversão de emoticons, os três clientes têm um seletor visual de emojis, em forma de pop-up com opções em grade, inspirado no WhatsApp. O seletor mostra os 48 emojis de `recursos/emojis.json`, lista usada pelos três clientes, e a escolha de um emoji o insere no campo de texto. No Desktop, o `SeletorEmojis` é um `JPopupMenu` com `GridLayout` de 6 linhas por 8 colunas, e a inserção ocorre na posição do cursor (`InsercaoEmoji`). Na Web, a grade é feita com CSS Grid, com 8 colunas. No Python, a janela "Emojis" também usa uma grade de 6 por 8.

A conversão de emoticons e o seletor são as duas formas de inserir emojis. As duas resultam no mesmo tipo de conteúdo, um texto Unicode que é cifrado e enviado como qualquer outra mensagem.

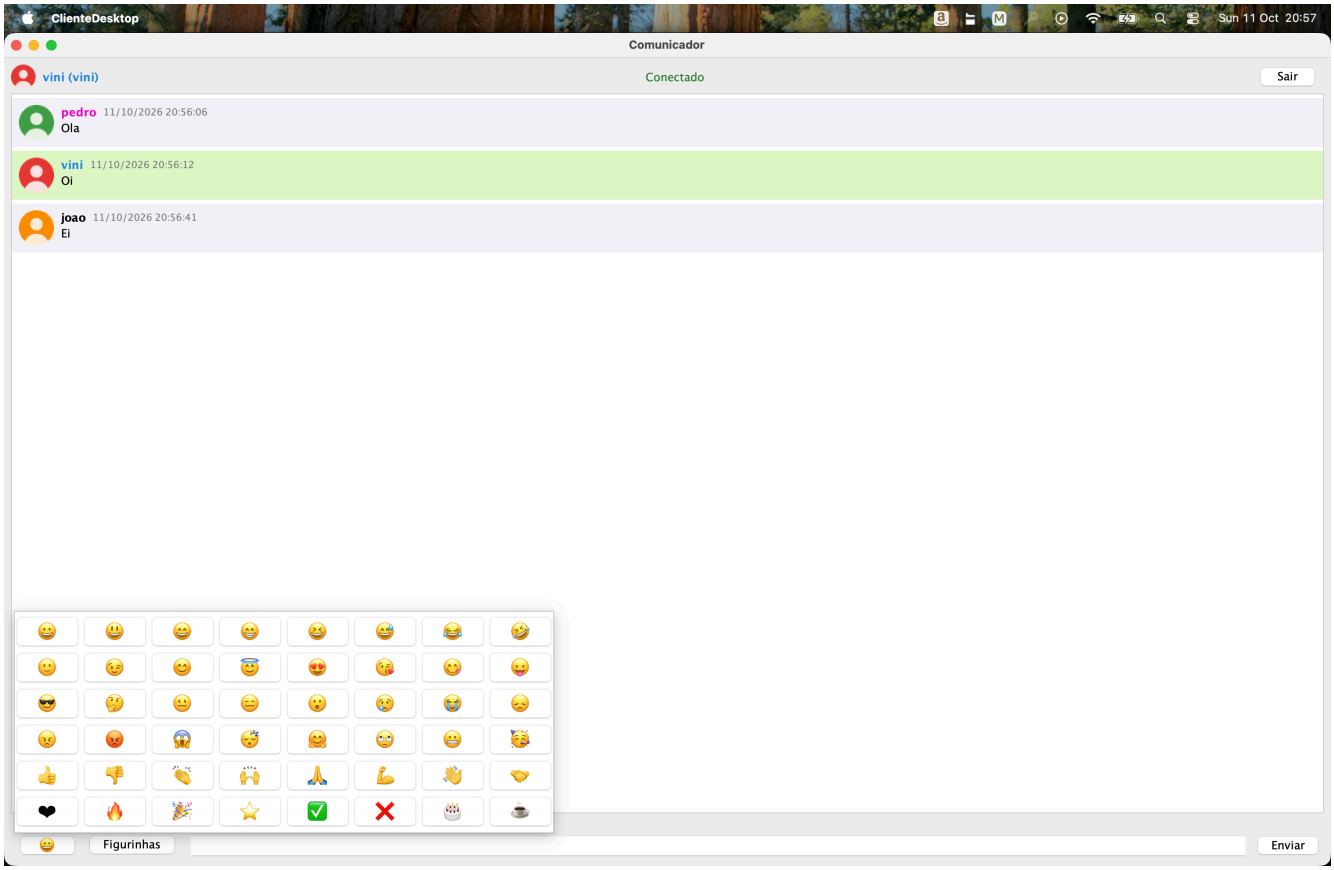


Figura 5: seletor de emojis do Cliente_Desktop (GridLayout).

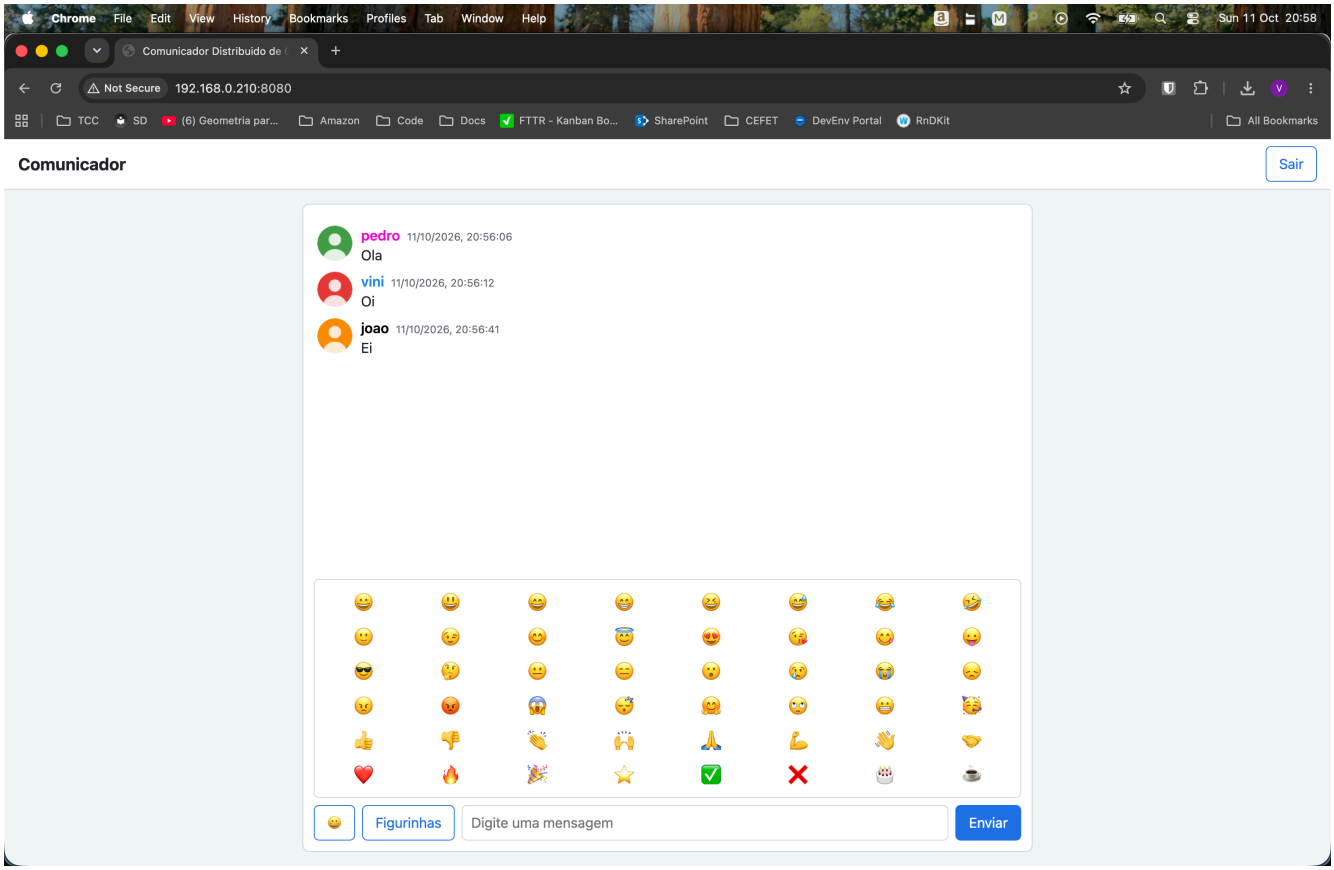


Figura 6: seletor de emojis do Cliente_Web (CSS Grid).

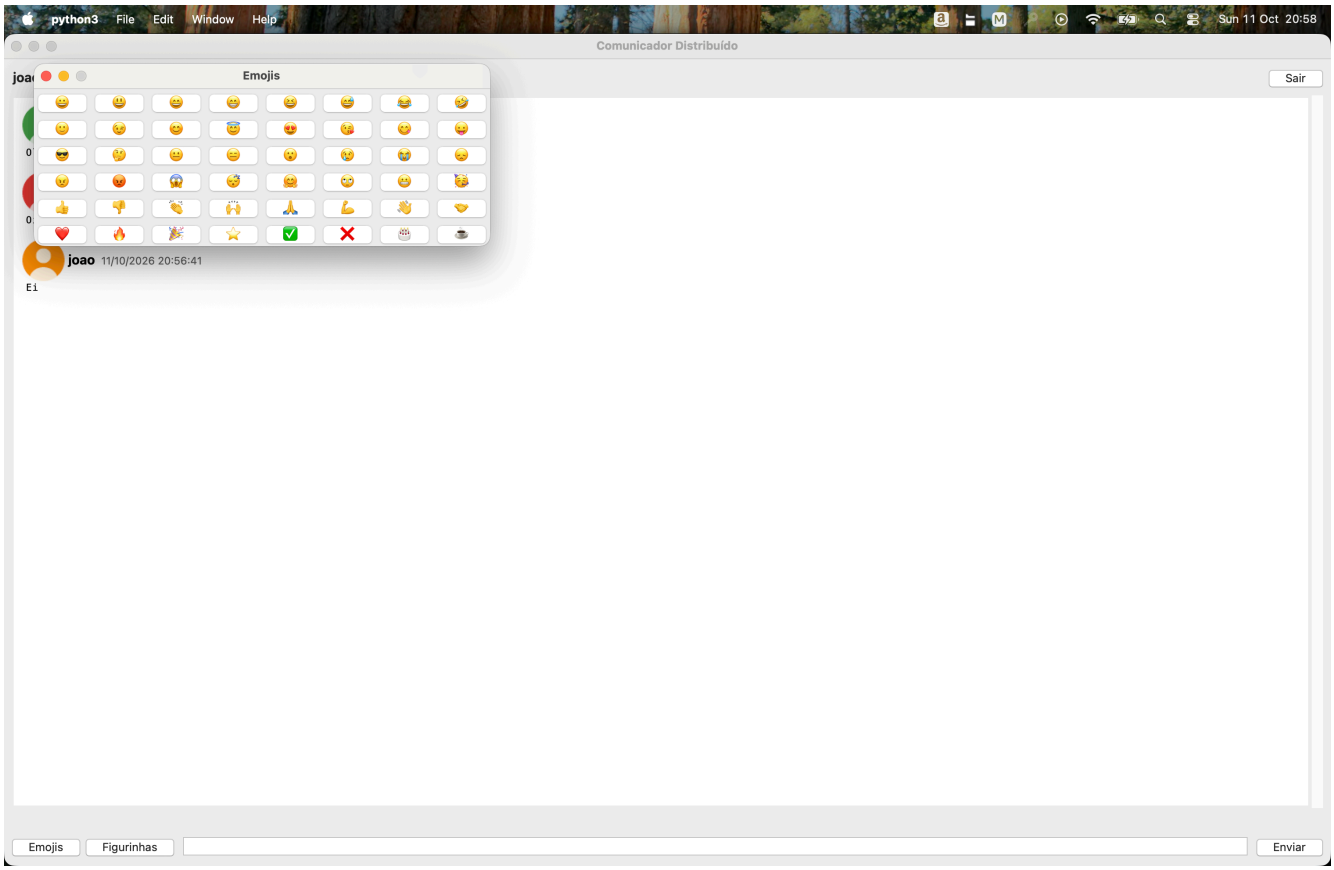


Figura 7: seletor de emojis do Cliente_Terceiro.

5.5 Figurinhas

Uma figurinha é uma imagem PNG identificada por `fig_NN`, onde `NN` tem dois dígitos. Existem 12 em `recursos/figurinhas/`, de `fig_01` a `fig_12`. O seletor de figurinhas é uma grade de miniaturas. Ao clicar em uma delas, o cliente envia na hora uma mensagem do tipo `figurinha`, sem passar pelo campo de texto e sem converter emoticons.

Cliente	Seletor
Desktop	`SeletorFigurinhas`, um `JPopupMenu` com `GridLayout` de 4 colunas e miniaturas de 64 px
Web	`<div id="seletor-figurinhas">`, com CSS Grid `repeat(auto-fill, minmax(4.5rem, 1fr))` e um `<button>` com `<img>` por figurinha
Python	`Toplevel` chamado "Figurinhas", com `Button.grid`, 4 colunas e miniaturas de até 56 px. Fecha ao escolher

Representação no protocolo e nos repositórios

O envio é uma operação `send_message` com `tipo` igual a `figurinha`. O `conteudo` é o identificador `fig_NN` cifrado com a chave do canal, exatamente como um texto é cifrado. A imagem nunca trafega, pois cada cliente já tem os PNG localmente.

Camada	Como a figurinha aparece
Protocolo (`send_message`, `message` e `history`)	`"tipo":"figurinha"` e `"conteudo"` igual à cifra (chave de trânsito) de `fig_03`
Repositório de conversas do servidor	`"tipo":"figurinha"` e `"conteudo"` igual à cifra (chave do repositório) de `fig_03`
Histórico local do cliente	`"tipo":"figurinha"` e `"conteudo"` igual à cifra (chave do histórico) de `fig_03`
Exibição	O cliente decifra, procura `fig_03.png` e mostra a imagem. Se o arquivo não existe, mostra um marcador textual

O Middleware decifra o conteúdo apenas em variável local e exige o padrão `^fig_[0-9]{2}$`. Se não bater, responde `CONTEUDO_INVALIDO` e não grava nada. Ele valida o padrão, não a existência do arquivo. O `tipo` fica em claro e revela que uma figurinha foi enviada, mas não qual.

Cliente	Marcador de figurinha sem imagem
Desktop	`[figurinha fig_99]`
Web	`[figurinha fig_99]`
Python	`[figurinha: fig_99]` (identificador fora do padrão: `[figurinha: desconhecida]`)

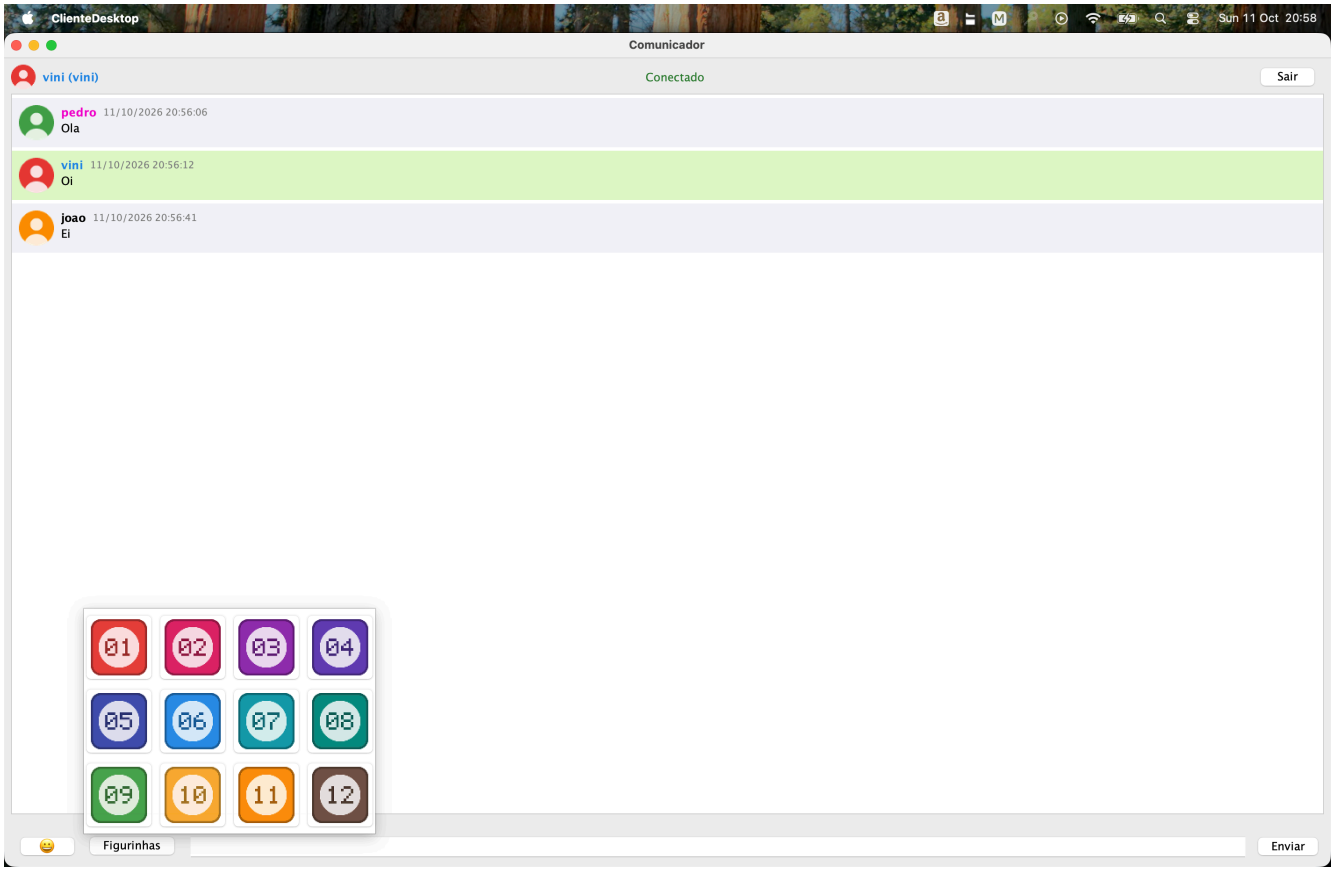


Figura 8: envio de figurinha e exibição nos três clientes.

---

6. Persistência por dispositivo

Cada dispositivo mantém um repositório local próprio, com as conversas enviadas e recebidas naquele cliente, inclusive as recuperadas do servidor. O Middleware guarda só os últimos sete dias, mas o histórico local de cada dispositivo continua com tudo o que aquele dispositivo recebeu ou enviou.

6.1 Onde fica o histórico em cada cliente

Cliente	Onde fica
Cliente_Desktop	`<raiz>/.comunicador/desktop/<login>/historico.txt` e `historico.key`
Cliente_Terceiro	`<raiz>/.comunicador/python/<login>/historico.txt` e `historico.key`
Cliente_Web	IndexedDB do navegador, banco `comunicador_<login>`

Desktop e Python. Há um diretório por usuário dentro de um diretório por cliente. Os dois clientes usam pastas diferentes (`desktop` e `python`), então um mesmo usuário que use os dois no mesmo computador tem dois históricos locais independentes. O diretório é criado com permissão `rwx------` e os arquivos com `rw-------`, quando o sistema oferece permissões POSIX. O `<login>` precisa casar com `^[a-z0-9_]{3,20}$`, o que impede que um login forjado como `../x` leve a gravação para fora do diretório do histórico.

Web. O banco se chama `comunicador_<login>` e tem dois object stores:

Object store	Chave	Conteúdo
`mensagens`	`mid`, com índice em `ts`	Um registro por mensagem, com o `conteudo` cifrado
`meta`	`historico_key` e `dispositivo`	A chave do histórico (64 hexadecimais) e o UUID do dispositivo

6.2 Formato do registro

O registro local tem os mesmos campos do repositório do servidor, mas com o perfil do remetente recebido do servidor e o `conteudo` cifrado com a chave do histórico.

```
{"mid":"6c1f0f0e-8f6b-4f0a-9d8f-3a1b7f7a9c11","remetente":{"login":"ana","apelido":"Ana","avatar":"av_03","cor":"#1E88E5"},"ts":"2025-
        03-14T18:22:05.431Z","tipo":"texto","conteudo":"<cifra com a chave do histórico>"}
```

Campo	Significado
`mid`	Identificador estável da mensagem, usado como chave de idempotência
`remetente`	`login`, `apelido`, `avatar` e `cor` do remetente (metadados em claro)
`ts`	Data e hora da mensagem, ISO-8601 em UTC com milissegundos
`tipo`	`texto` ou `figurinha`
`conteudo`	Cifra manual do conteúdo em Base64. Para `texto` é o texto (com emojis) e para `figurinha` é o identificador. Os dois são cifrados do mesmo modo

Nos clientes em arquivo, `historico.txt` tem um JSON por linha. Na Web, o mesmo objeto é guardado direto no object store. Não estão cifrados `mid`, `ts`, `tipo` e o perfil do remetente. Esses metadados são necessários para ordenar, deduplicar e exibir a mensagem, e já são conhecidos pelo servidor. O conteúdo da conversa, que é o dado sensível, nunca fica em claro.

6.3 Chave do histórico

A chave do histórico protege o `conteudo` no histórico local e é distinta da chave de trânsito e da chave do repositório. Ela é gerada na primeira abertura do histórico daquele usuário naquele dispositivo. O cliente sorteia 32 bytes aleatórios com um gerador do sistema (`SecureRandom` em Java, `secrets` em Python e `crypto.getRandomValues` na Web) e, se o valor coincidir com a chave de trânsito, sorteia de novo. Há uma chave por combinação de cliente, dispositivo e usuário, e cada diretório ou banco tem a sua.

A chave nunca trafega: não é enviada ao Middleware nem a outro cliente, e só quem a gerou consegue ler o histórico. Nos clientes em arquivo, ela fica em `historico.key` (64 hexadecimais), no mesmo diretório do histórico. Na Web, fica no object store `meta` do mesmo banco. No Desktop e no Python, o arquivo da chave é criado de forma exclusiva (arquivo temporário seguido de hard link), para que dois processos abrindo ao mesmo tempo não terminem com chaves diferentes. Na Web, a chave é lida e criada na mesma transação do IndexedDB.

6.4 Incorporação sem duplicar

O cliente incorpora ao histórico local tanto as mensagens que chegam em tempo real quanto as do histórico dos últimos sete dias que o Middleware envia ao entrar na sala. A operação é idempotente: incorporar o mesmo conjunto uma ou várias vezes produz o mesmo histórico, sem mensagens duplicadas. O identificador estável é o `mid`.

Cada mensagem do lote passa por três verificações. Se está marcada como ilegível ou não tem os campos obrigatórios, não é gravada e é contada como `ilegiveis` ou `invalidas`. Se o `mid` já existe no histórico, ou apareceu antes no mesmo lote, é ignorada e contada como `existentes`, e vale a primeira ocorrência. Caso contrário, o cliente decifra o `conteudo` com a chave de trânsito, cifra de novo com a chave do histórico (com um nonce novo) e grava. Se a decifragem falhar, a mensagem é contada como `ilegiveis` e não é gravada.

O resultado é um resumo com `adicionadas`, `existentes`, `ilegiveis` e `invalidas`. A forma de reconhecer o `mid` já guardado varia por cliente:

Cliente	Como reconhece o `mid` já guardado	Como grava
Desktop	Mapa `mid` para registro, carregado do arquivo na abertura	Anexa só os novos, em uma única escrita, com `force` (fsync)
Python	Mapa `mid` para registro, carregado do arquivo na abertura	Anexa só os novos, em uma única escrita, com `fsync`
Web	O `add` do IndexedDB falha com `ConstraintError` quando a chave existe, e o erro é tratado como "já existe"	Uma transação de escrita por lote

6.5 Persistência na Web

O Cliente_Web usa IndexedDB, a persistência do navegador que aceita volume maior e dados estruturados. O IndexedDB é isolado por origem (esquema, host e porta) e por perfil do navegador. Isso tem consequências diretas:

Situação	Resultado
Fechar e reabrir a aba ou o navegador	O histórico continua disponível
Outra origem, por exemplo `http://localhost:8080` e `http://192.168.0.10:8080`	São dois históricos diferentes, mesmo no mesmo computador
Outro perfil do navegador, ou outro navegador, no mesmo computador	Histórico diferente (outro dispositivo para o sistema)
Janela anônima ou privada	Em geral o histórico é descartado ao fechar a janela
Limpar dados do site	Perde o histórico e a chave do histórico, que ficam no mesmo banco. O que ainda está na janela de 7 dias volta do servidor na próxima entrada
Cliente_Web e Cliente_Desktop ou Python no mesmo computador	Históricos independentes, em lugares diferentes

O escopo é o de dispositivo e perfil do navegador. Não há sincronização entre dispositivos nem entre navegadores. O Cliente_Web não pede armazenamento persistente, então o navegador pode apagar dados de sites em situações de pouco espaço em disco.

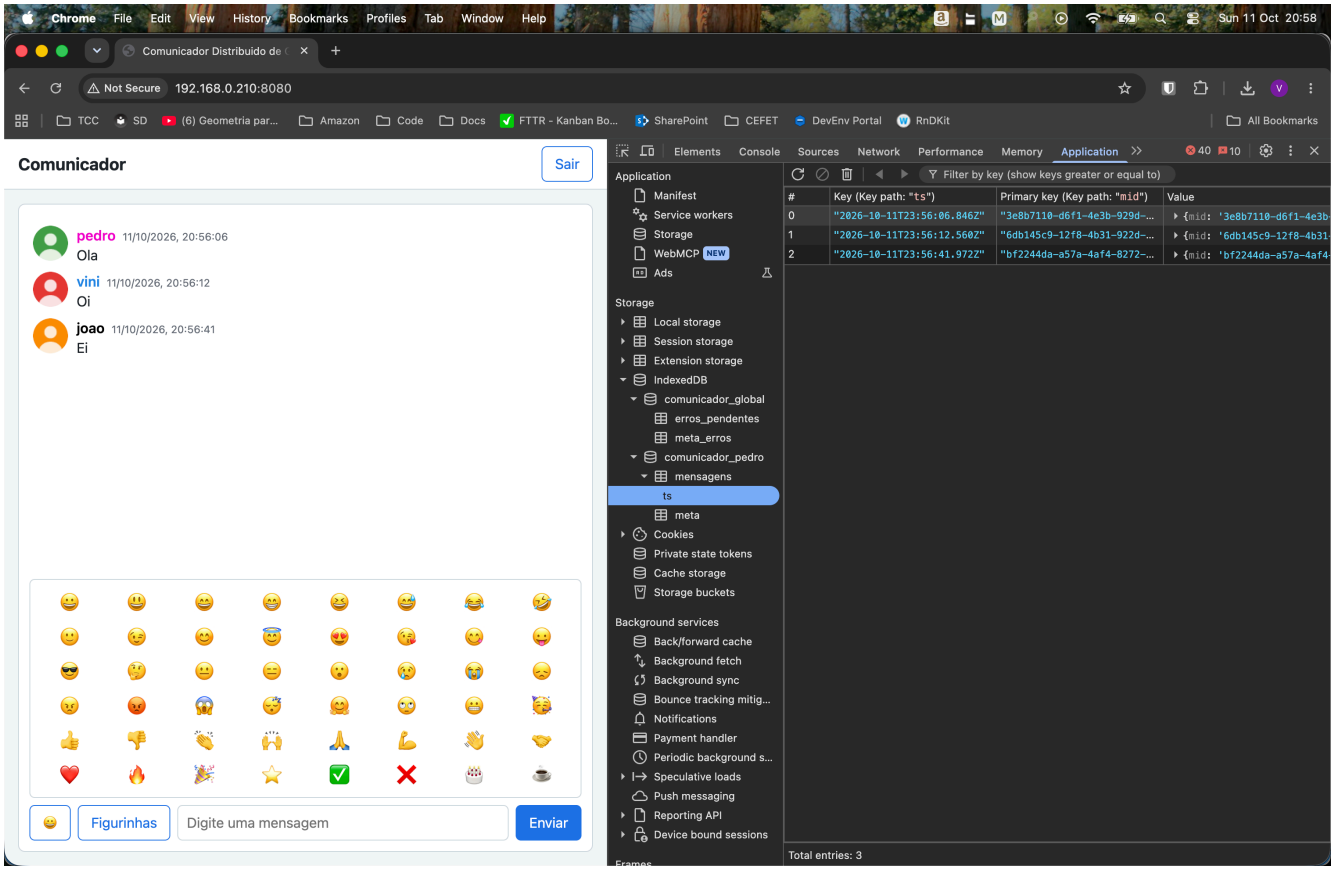


Figura 9: histórico local da Web em IndexedDB.

6.6 Separação entre contas no mesmo dispositivo

Dois usuários distintos podem usar o mesmo computador, o mesmo navegador ou o mesmo perfil do sistema operacional. O histórico de cada um fica separado:

Cliente	Mecanismo de separação
Web	Um banco IndexedDB por login: `comunicador_ana`, `comunicador_bruno` e assim por diante. Cada banco tem os próprios `mensagens` e `meta`, inclusive a própria chave do histórico
Desktop	Um diretório por login: `<raiz>/.comunicador/desktop/ana/`, `<raiz>/.comunicador/desktop/bruno/`, cada um com o próprio `historico.txt` e `historico.key`
Python	O mesmo esquema, em `<raiz>/.comunicador/python/<login>/`

O banco ou o diretório é aberto com o login que acabou de se autenticar. O cliente não lista nem abre o histórico de outro login. Trocar de conta abre outro histórico, e o do usuário anterior fica intacto. Como a cifra verifica um MAC e cada conta tem uma chave própria, um `historico.txt` copiado para o diretório de outro login não decifra, e as mensagens aparecem como ilegíveis.

---

7. Repositórios do servidor, retenção e limpeza

7.1 Os cinco repositórios

O Middleware mantém cinco repositórios separados. Quatro são arquivos de texto com um objeto JSON por linha, no diretório configurado em `dados.dir` (padrão `dados/`). O quinto é o banco de dados.

Repositório	Onde fica	Conteúdo	Retenção
Banco de dados	PostgreSQL, tabela `contas` (`sql/01_schema.sql`)	Contas: login, apelido, hash da senha, avatar e cor	Não sofre limpeza. A conta existe até ser removida por quem administra o banco
Conversas	`dados/conversas.txt`	Mensagens com o `conteudo` cifrado (chave do repositório)	Janela móvel de 7 dias
Log de envios	`dados/logs/envios.txt`	Somente `ts` e `login` de cada mensagem enviada, sem conteúdo	12 meses de calendário
Log de erros	`dados/logs/erros.txt`	Exceções do Middleware e dos clientes	90 dias
Log de acessos	`dados/logs/acessos.txt`	Eventos `conexao`, `login_ok`, `login_recusado` e `desconexao`	90 dias

O banco guarda apenas contas e não substitui o TXT de conversas. A limpeza atua só nos quatro arquivos. O diretório `dados/` não é versionado.

7.2 Registro de conversa em JSON

O arquivo de conversas tem uma linha por mensagem aceita, com os campos nesta ordem:

```
{"mid":"6c1f0f0e-8f6b-4f0a-9d8f-3a1b7f7a9c11","remetente":"ana","ts":"2025-03-
        14T18:22:05.431Z","tipo":"texto","conteudo":"AQECAwQFBgcIuR41d3TWk2QTGYx4p1B+WrWVCA=="}
{"mid":"0b7d2a54-3c1e-4b8a-9f10-5d6e7f809a1b","remetente":"bia","ts":"2025-03-
        14T18:22:09.002Z","tipo":"figurinha","conteudo":"ASEiIyQlJicoRIuxhRzH70y97twm/yc="}
{"mid":"3a5c8e10-7d24-4c3b-8e61-2f9b0c4d7a52","remetente":"ana","ts":"2025-03-
        14T18:23:41.870Z","tipo":"texto","conteudo":"AQECAwQFBgcItB0mOnWREY779l2CCu+Sk+hO568taQglJbNwMA=="}
```

Os três exemplos usam a chave fictícia `ffeeddcc...00` no papel da chave do repositório. Os textos claros são `Ola, grupo!`, `fig_03` e `Bora! 😀🎉 café`. Num servidor real a chave é outra e o nonce é aleatório a cada mensagem.

Campo	Tipo	Descrição
`mid`	texto (UUID v4)	Identificador da mensagem, atribuído pelo Middleware
`remetente`	texto	Somente o login do remetente. O perfil (apelido, avatar e cor) é obtido da conta quando o histórico é enviado
`ts`	texto	Data e hora do servidor, ISO-8601 em UTC com milissegundos e `Z`. É o campo usado pela janela de 7 dias e pela limpeza
`tipo`	`texto` ou `figurinha`	Fica em claro porque o Middleware precisa dele para distribuir e exibir
`conteudo`	Base64	Cifra manual, com a chave do repositório, do texto ou do identificador da figurinha

Um emoji é um caractere Unicode comum dentro do texto claro, antes de cifrar, então a cifra trabalha sobre os bytes UTF-8 dele. Em nenhum ponto do repositório o emoji ou o identificador da figurinha aparece legível, pois o `conteudo` é sempre Base64. Para ler, o Middleware decifra `conteudo` com a chave do repositório e cifra de novo com a chave de trânsito, em memória, ao montar o histórico.

7.3 Retenção de cada repositório

Todos os limites são calculados pelo relógio do servidor, em UTC, a partir do campo `ts` de cada linha. Um registro com `ts` exatamente igual ao limite é preservado. Só são removidos os registros com `ts` estritamente anterior ao limite.

Repositório	Limite preservado	Configuração
Conversas	`ts` maior ou igual a `agora − 7 dias`	Fixa, não configurável
Log de envios	`ts` maior ou igual a `agora − 12 meses de calendário`	`retencao.envios.meses` (padrão 12)
Log de erros	`ts` maior ou igual a `agora − N dias`	`retencao.logs.dias` (padrão 90)
Log de acessos	`ts` maior ou igual a `agora − N dias`	`retencao.logs.dias` (padrão 90)

Janela móvel. Os 7 dias não são "a semana corrente". Em cada execução o limite é recalculado como `agora − 7 dias`, e a janela anda junto com o relógio do servidor. O relógio dos clientes não entra no cálculo.

Meses de calendário. Para o log de envios, o limite é obtido subtraindo meses de calendário do instante atual em UTC (`ZonedDateTime.minusMonths`). Em 15/03/2026 o limite é 15/03/2025. Se o dia não existir no mês de destino, o dia é ajustado para o fim desse mês. Cada linha guarda data e hora completas, o que permite identificar o momento dentro do ano.

Prazo adotado para os logs de erros e de acessos: 90 dias

O enunciado não fixa o prazo desses dois logs. Adotei 90 dias, aproximadamente três meses. O primeiro motivo é o diagnóstico: erros e problemas de acesso costumam ser investigados dias ou semanas depois de acontecerem, e um trimestre cobre um ciclo completo de desenvolvimento, teste e correção, inclusive falhas intermitentes que só reaparecem depois de semanas. O segundo é manter apenas o mínimo necessário. Os logs contêm identificadores de usuário,

sessão e dispositivo, endereço de rede (log de acessos) e pilhas de exceção (log de erros), e guardar esses dados por mais tempo do que o diagnóstico exige amplia o impacto de um vazamento sem benefício operacional.

Há ainda duas razões práticas. A limpeza reescreve o arquivo inteiro a cada execução, então limitar o prazo mantém os arquivos pequenos e a reescrita rápida. Além disso, o prazo é menor que o do log de envios (12 meses) porque este é o único registro com finalidade de auditoria de longo prazo, e ele não guarda conteúdo nem dados de rede.

O valor é configurável e vale para os dois logs ao mesmo tempo.

7.4 Quando a limpeza ocorre

A limpeza roda em dois momentos. O primeiro é a partida do Middleware: a classe `MiddlewareMain` executa a limpeza de forma síncrona, depois de montar os repositórios e antes de abrir qualquer porta, então nenhum cliente consegue entrar na sala enquanto ela não termina. Se algum arquivo falhar, o Middleware avisa em `stderr` e inicia assim mesmo, e os demais arquivos são limpos normalmente.

O segundo momento é periódico, enquanto o Middleware está em execução. O padrão é 60 minutos, configurável em `limpeza.intervalo.minutos`. O agendamento usa `scheduleWithFixedDelay`: o intervalo conta do fim de uma execução ao início da seguinte, e duas execuções nunca se sobrepõem. O corpo agendado captura toda exceção, pois uma exceção não tratada cancelaria as execuções seguintes sem aviso.

Tolerância entre execuções. Entre duas execuções (até uma hora no padrão), o arquivo de conversas pode conter registros que acabaram de vencer. Isso não viola a retenção do ponto de vista do cliente, porque o envio do histórico aplica o filtro `ts` maior ou igual a `agora − 7 dias` ao montar o que é enviado. Um cliente nunca recebe uma mensagem fora da janela de 7 dias, mesmo que ela ainda esteja no arquivo. O arquivo pode guardar registros vencidos por, no máximo, um intervalo de limpeza.

7.5 Configuração

Os valores ficam em `config/servidor.properties` (copiado de `config/servidor.properties.example`):

```
retencao.logs.dias=90
retencao.envios.meses=12
limpeza.intervalo.minutos=60
dados.dir=dados
```

Cada propriedade também pode ser sobreposta por variável de ambiente, com o nome da propriedade em maiúsculas e o ponto trocado por sublinhado (por exemplo `RETENCAO_LOGS_DIAS=30`). Os três valores numéricos precisam ser inteiros maiores que zero. Um valor inválido faz o Middleware recusar a partida com uma mensagem que aponta a propriedade errada. A janela de 7 dias das conversas não tem propriedade, porque faz parte do requisito.

7.6 Reescrita atômica e concorrência

Remover linhas no meio de um arquivo exige reescrevê-lo. Enquanto isso, o Middleware continua recebendo mensagens e eventos de log, então a reescrita não pode perder nem corromper registros. Cada arquivo é gerenciado por um `RepositorioArquivo`, que tem um `ReentrantLock` próprio. As operações `anexar`, `lerLinhas` e `reescrever` usam o mesmo lock, e a limpeza usa a mesma instância que o resto do Middleware.

A reescrita acontece toda dentro do lock, e as anexações concorrentes esperam e continuam depois. A limpeza lê todas as linhas e mantém, na ordem e com o texto originais, as que são um objeto JSON válido com `ts` válido e maior ou igual ao limite. As linhas mantidas são gravadas em `<arquivo>.tmp`, no mesmo diretório, com a gravação forçada em disco. Por fim, o arquivo é trocado com `Files.move(tmp, arquivo, ATOMIC_MOVE, REPLACE_EXISTING)`. Se o processo cair no meio, o arquivo original continua íntegro.

---

8. Monitoramento e logs

8.1 Três logs separados

O Middleware grava três logs, cada um em arquivo próprio, todos com um objeto JSON por linha e `ts` em ISO-8601 UTC com milissegundos e `Z`.

Log	Arquivo	Finalidade	Retenção
Acessos	`dados/logs/acessos.txt`	Conexões, autenticações bem-sucedidas ou recusadas e desconexões	90 dias
Envios	`dados/logs/envios.txt`	Usuário, data e horário de cada mensagem enviada, sem conteúdo	12 meses
Erros	`dados/logs/erros.txt`	Exceções do Middleware e dos clientes	90 dias

Nenhum dos logs recebe senha, nem texto de conversa, nem conteúdo cifrado de mensagem. Os registros de diagnóstico não viram cópias paralelas do histórico.

8.2 Log de acessos

Campos, nesta ordem: `ts`, `evento`, `login`, `sessao`, `dispositivo`, `cliente` e `endereco`. O campo `motivo` existe somente nos eventos `desconexao`. Campos sem valor saem como `null`.

Campo	Descrição
`evento`	`conexao`, `login_ok`, `login_recusado` ou `desconexao`

Campo	Descrição
`login`	Login envolvido, ou `null`. Em `login_recusado` só é gravado se casar com o formato de login válido, para não registrar uma senha digitada no campo de login
`sessao`	Identificador da sessão, que só existe depois do login
`dispositivo`	UUID do dispositivo informado no `login`, ou `null`
`cliente`	`desktop`, `web` ou `python`, ou `null`
`endereco`	Endereço IP remoto da conexão, sem a porta
`motivo`	Só em `desconexao`: `logout`, `fechada_pelo_cliente`, `timeout`, `fila_cheia` ou `falhas_de_login`

O evento `conexao` é gravado na primeira operação `register` ou `login` da conexão, uma única vez por conexão.

```
{"ts":"2025-03-
        14T18:20:10.950Z","evento":"conexao","login":null,"sessao":null,"dispositivo":null,"cliente":null,"endereco":"192.168.0.12"}
{"ts":"2025-03-14T18:20:11.002Z","evento":"login_ok","login":"ana","sessao":"3f9a1c0de2b44f6a8c7d5e4b3a291807","dispositivo":"b1c2d3e4-
        0000-4000-8000-000000000001","cliente":"desktop","endereco":"192.168.0.12"}
{"ts":"2025-03-14T18:21:02.310Z","evento":"login_recusado","login":"ana","sessao":null,"dispositivo":"b1c2d3e4-0000-4000-8000-
        000000000001","cliente":"desktop","endereco":"192.168.0.12"}
{"ts":"2025-03-
        14T18:40:00.000Z","evento":"desconexao","login":"ana","sessao":"3f9a1c0de2b44f6a8c7d5e4b3a291807","dispositivo":"b1c2d3e4-
        0000-4000-8000-000000000001","cliente":"desktop","endereco":"192.168.0.12","motivo":"logout"}
```

8.3 Log de envios

Cada linha tem exatamente dois campos, `ts` e `login`, gravados a cada mensagem aceita. Não há `mid`, tipo, texto, figurinha, emoji nem conteúdo cifrado:

```
{"ts":"2025-03-14T18:22:05.431Z","login":"ana"}
```

O `ts` é o mesmo da mensagem correspondente. Como o `ts` é completo, ele identifica o dia e o horário dentro do ano, o que permite consultar o histórico de envios dos últimos 12 meses por usuário. Se a gravação nesse log falhar depois de a mensagem já estar persistida, o erro vai ao log de erros e a mensagem é distribuída mesmo assim, para que repositório e salas não divirjam.

8.4 Log de erros

Cada linha tem 11 campos, todos sempre presentes (os sem valor saem como `null`), nesta ordem:

Campo	Descrição
`ts`	Data e hora do evento. Do cliente quando `origem` é `cliente` (limitada ao instante atual do servidor, se o relógio do cliente estiver adiantado). Do servidor quando é `middleware`
`origem`	`middleware` ou `cliente`
`modulo`	Módulo de origem, por exemplo `OperacoesConversa.send_message`
`tipo`	Tipo da exceção (no Middleware, o nome simples da classe)
`descricao`	Descrição, mascarada e truncada em 2000 caracteres
`pilha`	Rastreamento da pilha, mascarado e truncado em 8000 caracteres
`sessao`	Identificador da sessão, ou `null` antes do login
`dispositivo`	UUID do dispositivo, ou `null`
`cliente`	`desktop`, `web` ou `python`, ou `null`
`evid`	Identificador do evento no buffer offline do cliente, ou `null` para eventos do Middleware
`descartados`	Quantidade de eventos descartados pelo buffer offline desde o último envio, ou `0`

Exemplo de evento do Middleware (falha de decifragem em `send_message`, com a pilha abreviada). O texto da exceção é fixo e não contém nada recebido:

```
{"ts":"2025-03-
        14T18:30:00.100Z","origem":"middleware","modulo":"OperacoesConversa.send_message","tipo":"DecifragemFalhouException","descricao":
        ao decifrar o campo conteudo","pilha":"br.cefet.sd.comum.cifra.DecifragemFalhouException: Falha ao decifrar o campo conteudo\n\ta
        ...","sessao":"3f9a1c0de2b44f6a8c7d5e4b3a291807","dispositivo":null,"cliente":null,"evid":null,"descartados":0}
```

Exemplo de evento de cliente, gravado por uma conexão ainda sem login:

```
{"ts":"2025-03-14T18:30:00.100Z","origem":"cliente","modulo":"conexao.js","tipo":"TypeError","descricao":"falha ao
        enviar","pilha":"TypeError: x\n  at conexao.js:10","sessao":null,"dispositivo":"b1c2d3e4-0000-4000-8000-
        000000000001","cliente":"web","evid":"0e9d1c2b3a4f","descartados":0}
```

8.5 Mascaramento de segredos

A função de mascaramento existe em Java, JavaScript e Python com o mesmo comportamento. Ela é aplicada a `descricao` e `pilha` no cliente antes de enviar e de novo no Middleware antes de gravar, sempre antes do truncamento, para que um segredo cortado ao meio não escape. A primeira regra troca por `***` o valor que vem depois de `senha`, `password`, `pwd`, `chave`, `key`, `token` ou `secret`, nas formas `nome=valor`, `nome: valor` e `"nome":"valor"`. A segunda troca por `[REDIGIDO]` as sequências Base64 com 24 ou mais caracteres e as sequências hexadecimais com 32 ou mais, o que cobre conteúdo cifrado e chaves.

Além das regras, há uma convenção: nenhuma exceção do projeto inclui o texto de uma mensagem ou de uma senha. O mascaramento não reconhece texto de conversa arbitrário dentro de uma exceção de biblioteca, e essa convenção é a defesa nesse caso.

Um efeito colateral conhecido: a regra de Base64 também pode mascarar nomes longos de classes Java na pilha. É um falso positivo conservador, que nunca vaza dado e só prejudica a leitura da pilha.

8.6 Exceções dos clientes enviadas ao servidor

Cada cliente tem um reportador central de erros que recebe as exceções capturadas e as não tratadas:

Cliente	Captura global
Desktop	`Thread.setDefaultUncaughtExceptionHandler` e o tratamento da thread de eventos do Swing
Web	`window.onerror` e `unhandledrejection`
Python	`sys.excepthook`, `threading.excepthook` e `Tk.report_callback_exception`

O reportador mascara os segredos e envia o evento ao Middleware com a operação `error_report`, que grava no log central de erros com `origem` igual a `cliente`, data e hora, módulo, identificação do dispositivo e do cliente, tipo da exceção, descrição e pilha. Erros esperados de requisição, como "login inválido", são mostrados ao usuário e não viram evento. A operação `error_report` não exige login, porque falhas acontecem também na tela de login.

8.7 Cliente sem conexão e proteção contra laço de reenvio

Se o cliente estiver sem conexão ou o envio falhar, o evento fica no buffer offline de erros, que é retido localmente e reenviado quando a conexão volta.

Cliente	Onde fica o buffer
Desktop	`<raiz>/.comunicador/desktop/erros_pendentes.txt`
Python	`<raiz>/.comunicador/python/erros_pendentes.txt`
Web	IndexedDB, banco `comunicador_global`

O buffer guarda no máximo 100 eventos e 256 KiB. Ao exceder, o evento mais antigo é descartado e o descarte é contado, e o contador `descartados` segue no próximo envio. Cada evento tem no máximo 5 tentativas, e a falta de conexão não conta como tentativa.

O buffer é esvaziado depois de cada login (inclusive o relogin automático após uma reconexão) e a cada 60 s enquanto houver itens. Cada requisição leva até 20 eventos, e o cliente remove do buffer somente os `evid` que o Middleware devolve em `aceitos`. Como o Middleware lembra os últimos 10.000 `evid` e confirma um reenvio sem gravar de novo, não há duplicatas no log.

Para evitar o laço "falha no envio gera erro, que gera outro envio", uma marca por thread (`emReporte`, aplicada com `try/finally`) impede a reentrância. Enquanto um reporte está em andamento, qualquer erro novo vai só para o console local e não cria evento.

---

9. Criptografia manual

A cifra manual é uma cifra de fluxo com autenticação por MAC. Ela cifra primeiro e calcula o MAC sobre o texto cifrado (*encrypt-then-MAC*). Foi implementada à mão nas três linguagens do sistema: Java (módulo `comum`, usado pelo Middleware e pelo Cliente_Desktop), JavaScript (Cliente_Web) e Python (Cliente_Terceiro).

O algoritmo usa apenas aritmética de inteiros de 32 bits sem sinal (soma, XOR, rotação e multiplicação módulo 2^32), que dá o mesmo resultado nas três linguagens. Nenhuma função ou biblioteca de cifragem é usada, nem recurso de criptografia do banco de dados. As únicas funções prontas são de codificação (UTF-8 e Base64) e uma fonte de números aleatórios do sistema operacional, usada só para sortear o nonce e gerar chaves. O resultado vira texto em Base64 apenas para trafegar em JSON e ser gravado em arquivos TXT. O Base64, isoladamente, é codificação e não faz parte da proteção.

O sistema aplica a mesma cifra em três canais, cada um com sua chave:

Canal protegido	Chave	O que protege
Trânsito (socket TCP e WebSocket)	Chave de trânsito	`conteudo` de `send_message`, `message` e `history`, e o campo `dados` de `register` e `login`
Repositório de conversas do servidor (`dados/conversas.txt`)	Chave do repositório	`conteudo` de cada registro
Histórico local de cada cliente	Chave do histórico	`conteudo` de cada registro

O Middleware decifra uma mensagem apenas em memória e a cifra de novo com a chave do canal de destino antes de transmitir ou gravar.

9.1 A chave

A chave tem 32 bytes (256 bits). Na configuração, ela é escrita como 64 caracteres hexadecimais, em que cada par de dígitos é um byte. A chave de trânsito e a do repositório vêm de `scripts/gerar_chave.py` (`python3 scripts/gerar_chave.py --par`), que sorteia 32 bytes com o gerador do sistema operacional. A chave do histórico é sorteada pelo próprio cliente na primeira execução (`SecureRandom`, `secrets` ou `crypto.getRandomValues`). O Middleware recusa iniciar se `chave.transito` e `chave.repositorio` forem iguais ou não tiverem 64 hexadecimais. Em Java, `Chave.toString()` devolve `Chave[***]`, então a chave não vai parar em log nem em exceção por descuido.

O nonce tem 8 bytes aleatórios por mensagem, sorteados a cada cifragem e guardados em claro dentro do resultado. Ele não é segredo. Sua função é garantir que o mesmo texto, cifrado duas vezes com a mesma chave, produza resultados diferentes.

9.2 Formato dos bytes cifrados

```
+----------+----------------+----------------------+--------------------+
| versão   | nonce          | texto cifrado        | tag (MAC)          |
| 1 byte   | 8 bytes        | n bytes (n = UTF-8)  | 8 bytes            |
| 0x01     | aleatório      | texto XOR keystream  | ver passo D        |
+----------+----------------+----------------------+--------------------+
```

O formato final é `Base64( versão | nonce | texto cifrado | tag )`, com o alfabeto padrão e preenchimento `=`. O tamanho mínimo decodificado é 17 bytes (texto vazio). Por isso todo `dados` ou `conteudo` do tráfego começa com `A` (versão `0x01` mais o início do nonce).

9.3 Primitivas de 32 bits

Todas as operações são módulo 2^32. A notação `rotl(x,k)` é a rotação de `k` bits à esquerda, `imul(a,b)` é o produto módulo 2^32, e `>>` é o deslocamento lógico (sem sinal) à direita.

Operação	Java	JavaScript	Python
`rotl(x,k)`	`Integer.rotateLeft(x,k)`	`((x<<k)|(x>>>(32-k)))>>>0`	`((x<<k)|(x>>(32-k)))&0xFFFFFFFF`
`imul(a,b)`	`a*b` (tipo `int`)	`Math.imul(a,b)>>>0`	`(a*b)&0xFFFFFFFF`
soma	`a+b` (tipo `int`)	`(a+b)>>>0`	`(a+b)&0xFFFFFFFF`
deslocamento sem sinal	`x>>>k`	`x>>>k`	`x>>k` (valores já mascarados)

O `>>>0` do JavaScript e a máscara `&0xFFFFFFFF` do Python reproduzem o comportamento natural do `int` de 32 bits do Java. É isso que mantém as três implementações idênticas byte a byte.

9.4 O algoritmo: passos A a D

Passo A: `absorve(h, dados)`

Mistura bytes em um estado `h` de quatro palavras de 32 bits.

```
para i de 0 até len(dados)-1:
    j = i AND 3
    x = h[j] XOR dados[i]
    x = rotl(x, 5)
    x = imul(x, 0x9E3779B1)
    h[j] = x
    k = (j + 1) AND 3
    h[k] = h[k] + (x >> 13)
```

Passo B: `mistura(h)`

Difusão final, com 8 rodadas.

```
repita 8 vezes:
    para j de 0 até 3:
        x = h[j] XOR (h[(j + 3) AND 3] >> 16)
        x = imul(x, 0x85EBCA6B)
        x = rotl(x, 13) XOR h[(j + 1) AND 3]
        h[j] = x
```

Passo C: gerador de keystream

O estado vem da chave e do nonce. A saída é uma sequência de palavras de 32 bits.

```
h = [0x243F6A88, 0x85A308D3, 0x13198A2E, 0x03707344]
absorve(h, chave || nonce)
mistura(h)
s = h    (se s for todo zero, s[0] = 1)
descarte 8 saídas de proximo()                  // aquecimento

proximo():                                      // estrutura xorshift com saída misturada
    r = imul( rotl( imul(s[1], 5), 7 ), 9 )
    t = s[1] << 9
    s[2] ^= s[0]    s[3] ^= s[1]    s[1] ^= s[2]    s[0] ^= s[3]
    s[2] ^= t       s[3] = rotl(s[3], 11)
    devolve r
```

O keystream é a concatenação das saídas de `proximo()`, cada uma escrita em 4 bytes little-endian (byte menos significativo primeiro), truncada ao tamanho do texto.

Passo D: MAC manual de 8 bytes

Calculado sobre versão, nonce e texto cifrado.

```
h = [0x452821E6, 0x38D01377, 0xBE5466CF, 0x34E90C6C]
absorve(h, 0x4D || chave)                       // 0x4D = 'M', separa domínios
absorve(h, versão || nonce || texto_cifrado)
absorve(h, len(texto_cifrado) em 4 bytes big-endian)
mistura(h)
tag = big-endian32(h[0] XOR h[2]) || big-endian32(h[1] XOR h[3])
```

Constantes

Constante	Valor	Uso
Versão	`0x01`	Primeiro byte do bloco. Qualquer outro valor é recusado
Estado inicial do gerador	`243F6A88 85A308D3 13198A2E 03707344`	Passo C
Estado inicial do MAC	`452821E6 38D01377 BE5466CF 34E90C6C`	Passo D
Multiplicador de `absorve`	`0x9E3779B1`	Passo A
Multiplicador de `mistura`	`0x85EBCA6B`	Passo B
Rodadas de `mistura`	8	Passo B
Rotações	5 (`absorve`), 13 (`mistura`), 7 e 11 (`proximo`)	Passos A, B e C
Multiplicadores de `proximo`	5 e 9	Passo C
Aquecimento	8 saídas descartadas	Passo C
Prefixo de domínio do MAC	`0x4D` (letra M)	Passo D
Tamanhos	chave 32, nonce 8, tag 8	Formato

Os estados iniciais são dígitos hexadecimais de pi, escolhidos assim para mostrar que não há valor especial escondido. A constante `0x9E3779B1` vem da razão áurea e `0x85EBCA6B` é uma constante conhecida de misturadores de bits. É apenas uma escolha de projeto, não uma garantia de segurança.

9.5 Etapas de cifragem e decifragem

Cifragem: `cifrar(chave, texto)`

Primeiro, `pt` recebe os bytes UTF-8 do texto e `nonce` recebe 8 bytes aleatórios. Em seguida, `ks` recebe o keystream de `len(pt)` bytes (passo C, semeado por `chave || nonce`), e cada byte `ct[i]` é calculado como `pt[i]` XOR `ks[i]`. Depois, `tag` é calculada como o MAC sobre `versão`, `nonce` e `ct` (passo D). A saída é o Base64 de `0x01 || nonce || ct || tag`.

Para conferência, existe também `cifrarComNonce(chave, nonce, texto)`, a mesma função com o nonce informado. A função de produção apenas sorteia o nonce e a chama.

Decifragem: `decifrar(chave, b64)`

A decifragem começa decodificando o Base64 de forma estrita: alfabeto padrão, preenchimento `=` correto e nenhum espaço ou quebra de linha. Java e Python também recodificam o resultado e exigem que seja igual à entrada, e o JavaScript valida o formato com uma expressão regular antes de chamar `atob`. Qualquer falha lança `DecifragemFalhou`. Depois, o código exige comprimento mínimo de 17 bytes e primeiro byte `0x01`, e separa `nonce` (bytes 1 a 8), `ct` e `tag` (os 8 últimos bytes).

O passo seguinte recalcula o MAC e o compara com a `tag` em tempo constante (acumula o XOR de todos os bytes e só testa no fim). Qualquer diferença lança `DecifragemFalhou`. Esse passo detecta chave errada e adulteração e vem antes de qualquer decifragem. Só então `pt[i]` é calculado como `ct[i]` XOR `ks[i]`, e o resultado é decodificado como UTF-8 de forma estrita. Bytes inválidos lançam `DecifragemFalhou`.

Quando a decifragem falha, o Middleware rejeita o dado, responde `DECIFRAGEM_FALHOU` e grava no log de erros um texto fixo, sem o conteúdo recebido. A conexão e o sistema continuam em operação.

9.6 Representação dos bytes

Dado	Representação
Chave	32 bytes. Em configuração, 64 caracteres hexadecimais
Nonce	8 bytes brutos dentro do bloco. Nos exemplos, 16 hexadecimais (`0102030405060708`)
Texto de entrada	Cadeia Unicode convertida para bytes UTF-8
Saída do keystream	Palavras de 32 bits escritas em little-endian
Comprimento no MAC	4 bytes big-endian
Tag	Dois valores de 32 bits em big-endian, 8 bytes no total

Dado	Representação
Resultado	`versão | nonce | ct | tag` em Base64 padrão com `=`

9.7 Código da cifra

O código completo está em `comum/src/main/java/br/cefet/sd/comum/cifra/CifraManual.java` (Java), `cliente-web/js/cifra.js` (JavaScript) e `cliente-python/comunicador/nucleo/cifra.py` (Python). Os trechos abaixo mostram o núcleo em Java: os passos A, B e C e a função de cifragem. O passo D (MAC) segue o pseudocódigo da seção 9.4.

```java
// Passo A: mistura bytes no estado de quatro palavras.
private static void absorve(int[] h, byte[] dados) {
    for (int i = 0; i < dados.length; i++) {
        int j = i & 3;
        int x = h[j] ^ (dados[i] & 0xFF);
        x = Integer.rotateLeft(x, 5);
        x = x * 0x9E3779B1;
        h[j] = x;
        int k = (j + 1) & 3;
        h[k] = h[k] + (x >>> 13);
    }
}

// Passo B: difusão final, 8 rodadas.
private static void mistura(int[] h) {
    for (int r = 0; r < 8; r++) {
        for (int j = 0; j < 4; j++) {
            int x = h[j] ^ (h[(j + 3) & 3] >>> 16);
            x = x * 0x85EBCA6B;
            x = Integer.rotateLeft(x, 13) ^ h[(j + 1) & 3];
            h[j] = x;
        }
    }
}

// Passo C: uma saída do gerador (xorshift com saída misturada).
private static int proximo(int[] s) {
    int r = Integer.rotateLeft(s[1] * 5, 7) * 9;
    int t = s[1] << 9;
    s[2] ^= s[0];
    s[3] ^= s[1];
    s[1] ^= s[2];
    s[0] ^= s[3];
    s[2] ^= t;
    s[3] = Integer.rotateLeft(s[3], 11);
    return r;
}

// Cifragem: texto UTF-8 XOR keystream, seguido do MAC.
public static String cifrarComNonce(Chave k, byte[] nonce, String texto) {
    byte[] chave = k.bytesInternos();
    byte[] pt = paraUtf8(texto);
    byte[] ks = keystream(chave, nonce, pt.length);
    byte[] ct = new byte[pt.length];
    for (int i = 0; i < pt.length; i++) {
        ct[i] = (byte) (pt[i] ^ ks[i]);
    }
    byte[] tag = mac(chave, nonce, ct);
    // saída = Base64( 0x01 || nonce || ct || tag )
}
```

Em Java, o tipo `int` já faz a aritmética módulo 2^32. As outras duas linguagens reproduzem esse comportamento com máscaras, como mostra a tabela da seção 9.3. Em JavaScript, os passos A e C ficam assim:

```
function absorve(h, dados) {
  for (let i = 0; i < dados.length; i++) {
    const j = i & 3;
    let x = (h[j] ^ dados[i]) >>> 0;
    x = rotl(x, 5);
    x = imul(x, 0x9e3779b1);   // Math.imul(a, b) >>> 0
    h[j] = x;
    const k = (j + 1) & 3;
    h[k] = (h[k] + (x >>> 13)) >>> 0;
  }
}

const proximo = () => {
  const r = imul(rotl(imul(s[1], 5), 7), 9);
  const t = (s[1] << 9) >>> 0;
  s[2] = (s[2] ^ s[0]) >>> 0;  s[3] = (s[3] ^ s[1]) >>> 0;
  s[1] = (s[1] ^ s[2]) >>> 0;  s[0] = (s[0] ^ s[3]) >>> 0;
  s[2] = (s[2] ^ t) >>> 0;     s[3] = rotl(s[3], 11);
  return r;
};
```

Em Python, os mesmos passos usam a máscara `0xFFFFFFFF`:

```
def absorve(h, dados):
    for i, byte in enumerate(dados):
        j = i & 3
        x = _rotl(h[j] ^ byte, 5)
        x = _imul(x, 0x9E3779B1)   # (a * b) & 0xFFFFFFFF
        h[j] = x
        k = (j + 1) & 3
        h[k] = (h[k] + (x >> 13)) & 0xFFFFFFFF

def proximo(self):
    s = self._s
    r = _imul(_rotl(_imul(s[1], 5), 7), 9)
    t = (s[1] << 9) & 0xFFFFFFFF
    s[2] ^= s[0]; s[3] ^= s[1]; s[1] ^= s[2]; s[0] ^= s[3]
    s[2] ^= t;    s[3] = _rotl(s[3], 11)
    return r
```

9.8 Exemplo completo

O exemplo cifra a mensagem `Ola, grupo!` com a chave fictícia de trânsito `000102...1f` e o nonce `0102030405060708`. Em uso real, o nonce é sorteado a cada mensagem.

Primeiro, o texto vira bytes UTF-8: `4f 6c 61 2c 20 67 72 75 70 6f 21` (11 bytes). Em seguida, o gerador é semeado com `chave || nonce`. Partindo do estado inicial `243F6A88 85A308D3 13198A2E 03707344`, o passo A leva o estado a `4756c385 cf3fdcd0 a80cc103 9a649930`, e o passo B a `fc927f4a 2ed828d6 c16ee0a7 b536dbfa`. Depois das 8 saídas de aquecimento, o gerador produz o keystream, que é cortado em 11 bytes.

A aplicação da chave é o XOR byte a byte entre o texto e o keystream:

Byte	1	2	3	4	5	6	7	8	9	10	11
Texto	4f	6c	61	2c	20	67	72	75	70	6f	21
Keystream	8d	fc	44	9b	c7	a5	9e	6b	25	aa	6e
Cifrado	c2	90	25	b7	e7	c2	ec	1e	55	c5	4f

O MAC (passo D) é calculado sobre `01`, o nonce e o texto cifrado, e resulta na tag `ba15dc244ec63d87`. O bloco final tem 28 bytes:

```
01 | 0102030405060708 | c29025b7e7c2ec1e55c54f | ba15dc244ec63d87
```

Em Base64, ele vira `AQECAwQFBgcIwpAlt+fC7B5VxU+6FdwkTsY9hw==`, que é o valor enviado no campo `conteudo`.

Para recuperar a mensagem, o receptor decodifica o Base64 e confere que o primeiro byte é `01`. Depois separa o nonce, o texto cifrado e a tag, recalcula o MAC e compara com a tag recebida. Como os valores coincidem, ele regenera o mesmo keystream a partir de `chave || nonce` e aplica o XOR de novo. O resultado é `4f 6c 61 2c 20 67 72 75 70 6f 21`, que em UTF-8 é exatamente `Ola, grupo!`.

A cifra também recusa o que não deve aceitar. Se um único bit do texto cifrado for alterado, ou se a decifragem usar outra chave, o MAC recalculado não confere e a decifragem falha antes de qualquer XOR. Com a chave fictícia do repositório (`ffeeddcc...00`) e o mesmo nonce, a mesma mensagem produz `AQECAwQFBgcIuR41d3TWk2QTGYx4p1B+WrWVCA==`, um resultado diferente. Por isso um registro do arquivo de conversas não decifra com a chave de trânsito.

9.9 Conferência entre Java, JavaScript e Python

Para validar a interoperabilidade, as quatro entradas abaixo foram cifradas com o mesmo nonce (`0102030405060708`) nas três implementações. As três produziram exatamente o mesmo Base64, e cada uma decifrou corretamente o valor produzido pelas outras. Os textos com acento e com emojis mostram que a cifra trata caracteres especiais apenas como bytes UTF-8.

Texto	Chave	Resultado (igual nas três)
`Ola, grupo!`	trânsito (`000102...1f`)	`AQECAwQFBgcIwpAlt+fC7B5VxU+6FdwkTsY9hw==`
`Ola, grupo!`	repositório (`ffeeddcc...00`)	`AQECAwQFBgcIuR41d3TWk2QTGYx4p1B+WrWVCA==`
`Atenção: reunião às 15h, coração é ñ ü ç`	trânsito	`AQECAwQFBgcIzIgh9QQCXchKkE6dycvC8eBsQO+oz+f7mXws9zvJ/A5lRWgjdv/Fj+SL+SrmTvmrLVtnpci4p8bgVA==`
`Bora! 😀🎉 café`	repositório	`AQECAwQFBgcItB0mOnWREY779l2CCu+Sk+hO568taQglJbNwMA==`

9.10 Distribuição das chaves

A chave nunca viaja pelo mesmo meio da mensagem que ela protege. A chave de trânsito é entregue fora do socket, e as outras duas nem existem na rede.

Chave	Origem	Como chega a quem a usa	Quem a conhece
Chave de trânsito	`scripts/gerar_chave.py` (32 bytes aleatórios)	Entrega manual, fora do socket. Servidor: `config/chaves.properties` (`chave.transito`). Desktop e Python: arquivo `<raiz>/.comunicador/chave_transito.txt`, copiado junto com a instalação. Navegador: o usuário digita a chave na tela de login (guardada no `localStorage`) ou abre um link com `#chave=<hex>`, e o fragmento nunca é enviado ao servidor HTTP e é apagado da URL após a leitura	Middleware e os três clientes
Chave do repositório	`scripts/gerar_chave.py`	Somente `config/chaves.properties` do servidor (`chave.repositorio`)	Somente o Middleware
Chave do histórico	Gerada pelo cliente na primeira execução, por cliente, dispositivo e usuário	Nunca trafega. Fica em `historico.key` (Desktop e Python) ou no object store `meta` do IndexedDB (Web)	Somente o cliente que a gerou

O arquivo `config/chaves.properties.example` traz valores fictícios e lembra que o arquivo real não deve ser versionado e deve ter permissão 600. Cada canal usa uma chave distinta, então um registro do arquivo de conversas não decifra com a chave de um histórico local nem com a de trânsito.

9.11 Módulos que conhecem cada chave e onde a mensagem fica legível

Não há criptografia de ponta a ponta. O Middleware precisa ler o texto para validar, gravar e redistribuir a mensagem, então ele é o único módulo do servidor que conhece as chaves de conversa. A tabela mostra o que cada módulo conhece e onde o texto aparece legível.

Módulo	Chaves que conhece	Onde a mensagem fica legível
Middleware	Trânsito e repositório	Somente na memória, durante o processamento. Decifra com a chave de trânsito, valida e cifra de novo com a do repositório (para gravar) e com a de trânsito (para distribuir). Nunca vai a disco nem a log em claro
Cliente_Desktop e Cliente_Terceiro	Trânsito e a própria chave do histórico	Na tela da sala e na memória do processo. Em disco, apenas cifrada
Cliente_Web	Trânsito e a própria chave do histórico	Na página e na memória do navegador. No IndexedDB, apenas cifrada
PostgreSQL	Nenhuma	Não recebe conteúdo de conversa, só contas
Servidor HTTP estático	Nenhuma	Serve apenas os arquivos do Cliente_Web
Logs	Nenhuma	Não contêm texto de conversa nem conteúdo cifrado

A senha também fica legível por um instante na memória do Middleware, o tempo de calcular ou conferir o hash, e é descartada em seguida (seção 9.15).

9.12 Metadados expostos e justificativa

Alguns campos precisam ficar em claro para o sistema funcionar. O enunciado pede que o uso e a exposição deles sejam justificados, e a tabela faz isso.

Campo	Onde aparece	Por que fica em claro	O que revela
`v`, `op` e `id`	Envelope do protocolo	O processador precisa deles para saber o que fazer antes de decifrar qualquer coisa	Qual operação foi pedida
`mid`	Protocolo, repositório e histórico local	É a chave de ordenação e de deduplicação	Nada sobre o conteúdo
`ts`	Protocolo, repositório e histórico local	Define a janela de 7 dias, a limpeza e a ordem de exibição	Quando houve uma mensagem
`tipo`	Protocolo, repositório e histórico local	O Middleware valida o conteúdo conforme o tipo, e o cliente decide se mostra texto ou imagem	Se foi texto ou figurinha, mas não qual
Remetente	Protocolo e histórico local (login, apelido, avatar e cor). No repositório, só o login	Necessário para exibir a mensagem, e o servidor já conhece esses dados	Quem escreveu

Esses dados já são conhecidos pelo servidor, e o dado sensível, o conteúdo da conversa, nunca fica em claro em nenhum canal. Um observador do tráfego ou dos arquivos vê quem escreveu e quando, mas não o que foi escrito. Os logs de auditoria seguem uma regra própria: o log de envios guarda apenas `ts` e `login`, e nenhum log recebe conteúdo de mensagem (seção 8.1).

9.13 Bytes, texto, emojis, figurinhas, imagens e acentos

A cifra opera sobre bytes. A interface pública recebe texto, e a conversão é sempre a mesma.

Dado	O que entra na cifra	Observação
Texto	Bytes UTF-8 do texto	Até 2000 pontos de código por mensagem
Emoji	Bytes UTF-8 do emoji, dentro do texto	Um emoji fora do plano básico ocupa 4 bytes e conta como 1 ponto de código
Figurinha	Bytes UTF-8 do identificador (`fig_03`)	Tratado como qualquer texto. O identificador é o único dado que circula
Imagem (PNG de figurinha ou avatar)	Nada. A imagem nunca trafega nem é cifrada	Cada cliente já tem o arquivo em `recursos/`. Só o identificador circula
Bytes arbitrários	Não usados pelo sistema	O núcleo aceitaria bytes. Para enviá-los pela API de texto seria preciso codificá-los antes, por exemplo em Base64

A decifragem faz o caminho inverso: bytes, depois decodificação UTF-8 estrita. Bytes que não formam UTF-8 válido são recusados, em vez de serem trocados por `?` ou por U+FFFD.

Emojis e figurinhas protegidos. Em nenhum ponto do protocolo, do repositório de conversas ou do histórico local o emoji ou o identificador da figurinha aparece em claro. O `conteudo` é sempre Base64. Quem lê o socket ou os arquivos vê `"tipo":"figurinha"` mais um Base64 e não descobre qual figurinha foi enviada. O apelido do remetente, por exemplo `Bia ✨`, não é cifrado, porque faz parte do perfil público.

Acentos e caracteres especiais. Acentos, emojis, quebras de linha, aspas e barras são só bytes UTF-8 para a cifra, então não há tratamento especial. Como exemplo, o texto `Bora! 😀🎉 café`, que mistura emojis e acento e ocupa 20 bytes em UTF-8, cifrado com a chave fictícia `ffeeddcc...00` e o nonce `0102030405060708`, resulta em `AQECAwQFBgcItB0mOnWREY779l2CCu+Sk+hO568taQglJbNwMA==`. Um cuidado da implementação é que o texto precisa ser Unicode válido. Uma cadeia com *surrogate* solitário (metade de um par UTF-16) não representa texto real, e as linguagens divergem nesse caso. O Java recusa na cifragem com `IllegalArgumentException`, o `TextEncoder` do JavaScript troca o caractere por U+FFFD e o Python lança `UnicodeEncodeError`. Esse caso fica fora do escopo.

9.14 Dado atravessando o socket cifrado e repositórios cifrados

Em trânsito. Todo `dados` (`register` e `login`) e todo `conteudo` (`send_message`, `message` e `history`) viaja cifrado com a chave de trânsito, pelo TCP e pelo WebSocket. Em claro ficam só os metadados do envelope (`v`, `op` e `id`) e do registro (`mid`, `ts`, `tipo` e o perfil do remetente, seção 6.2). Um quadro de `send_message` visto no socket tem este aspecto:

```
{"v":1,"op":"send_message","id":"m1","tipo":"texto","conteudo":"AQECAwQFBgcIwpAlt+fC7B5VxU+6FdwkTsY9hw=="}
```

Para capturar o tráfego do TCP, execute na máquina do servidor (ou em outra com acesso ao fluxo), enquanto os clientes trocam mensagens:

```
sudo tcpdump -i any -A -s 0 'tcp port 9000'
```

No Wireshark, o filtro equivalente é `tcp.port == 9000`, e a opção "Follow TCP Stream" mostra as linhas JSON. No navegador, abra as ferramentas de desenvolvimento, aba Rede (Network), filtre por WS e selecione a conexão para ver cada frame.

No repositório de conversas do servidor. O arquivo `dados/conversas.txt` guarda um JSON por linha com `mid`, `remetente`, `ts`, `tipo` e `conteudo` cifrado com a chave do repositório (seção 7.2).

No histórico local de cada cliente. O Desktop e o Python guardam `historico.txt` (um JSON por linha) e o Cliente_Web guarda o object store `mensagens` do IndexedDB. Em todos, o `conteudo` está cifrado com a chave do histórico do dispositivo e do usuário (seção 6.2).

```
head -n 3 dados/conversas.txt
head -n 2 .comunicador/desktop/<login>/historico.txt
```

9.15 Tratamento de senhas (separado da cifra)

A senha não usa a cifra manual para ser guardada. Os dois mecanismos são distintos. A cifra é reversível, com a chave certa, e serve às conversas. O tratamento de senhas é unidirecional, pois não existe operação para recuperar a senha a partir do que está guardado. O código está em `middleware/.../contas/TratamentoSenhas.java`.

Algoritmo

SHA-256 (`MessageDigest`, função de hash pronta, que não é função de cifragem), iterado `N` vezes, com um salt aleatório por conta. Sejam `salt` os 16 bytes brutos e `senha_utf8` a senha em UTF-8:

```
h = SHA256(salt || senha_utf8)                  // iteração 1
repita N−1 vezes:
    h = SHA256(h || salt || senha_utf8)
hash = hex_minúsculo(h)                         // 64 caracteres
```

O código Java correspondente:

```java
private static byte[] calcular(String senha, byte[] salt, int n) {
    MessageDigest sha = novoDigest();
    byte[] s = senha.getBytes(StandardCharsets.UTF_8);
    sha.update(salt);
    sha.update(s);
    byte[] h = sha.digest();
    for (int i = 1; i < n; i++) {
        sha.update(h);
        sha.update(salt);
        sha.update(s);
        h = sha.digest();
    }
    java.util.Arrays.fill(s, (byte) 0);
    return h;
}
```

Salt, iterações e formato armazenado

O salt tem 16 bytes aleatórios (`SecureRandom`) por conta, em hexadecimal minúsculo, então duas contas com a mesma senha têm hashes diferentes. O número de iterações padrão é `N = 100000`, configurável pela propriedade `senha.iteracoes`, e o valor usado em cada conta fica gravado no próprio registro. O valor guardado em `contas.senha_hash` tem quatro campos separados por `$`:

```
sha256i$<N>$<salt hex>$<hash hex>
```

Como `N` está no valor, a verificação não depende da configuração atual, e dá para mudar o padrão sem invalidar contas antigas.

Exemplo fictício, com a senha de demonstração `senha-de-teste-123`, o salt `00112233445566778899aabbccddeeff` e `N = 1000` apenas para o exemplo (a produção usa 100000). O valor foi calculado pela classe `TratamentoSenhas` (Java) e por `hashlib` (Python), e os dois deram o mesmo resultado:

```
sha256i$1000$00112233445566778899aabbccddeeff$61dc5649b3766a6e00e88c88f690b246881cb2edfeedf719e5d911ff114f5087
```

Cadastro e login

A senha trafega cifrada com a cifra manual e é guardada como hash. O cliente monta o JSON (`{"login":"...","senha":"..."}`), cifra com a chave de trânsito e envia em `dados`, de modo que login e senha não aparecem em claro no socket. O Middleware decifra `dados` em memória e, se a decifragem falhar, responde `DECIFRAGEM_FALHOU` sem gravar o conteúdo.

No cadastro, o Middleware valida os campos, gera um salt novo e grava `sha256i$N$salt$hash` na tabela `contas`, sem gravar a senha em texto. No login, busca a conta, recalcula o hash com o `N` e o salt do valor armazenado e compara com `MessageDigest.isEqual`, que leva tempo constante. Depois descarta a senha. Se a conta não existe, o Middleware chama `verificarContaInexistente`, que calcula o mesmo número de iterações contra um salt fixo. O tempo de resposta fica equivalente ao de uma conta real, e a resposta é `CREDENCIAIS_INVALIDAS` nos dois casos, então quem observa a resposta não descobre quais logins existem. Um valor armazenado malformado nunca causa exceção, e a verificação devolve "não confere". Nenhuma exceção da classe inclui a senha.

A senha em texto existe apenas em variável local do Middleware (apagada após o cálculo) e, no cliente, só na memória do processo para o relogin automático. Nunca vai para `localStorage`, IndexedDB ou arquivo.

Limite. SHA-256 iterado é mais fraco que bcrypt, scrypt ou Argon2, que não existem na biblioteca padrão do Java. A escolha mantém o projeto sem dependência de criptografia e serve ao objetivo didático. Para uso real, deve-se usar um algoritmo de derivação de senha próprio para esse fim.

---

10. Concorrência, reconexão e falhas

10.1 Concorrência no Middleware

Tema	Decisão	Por quê
Conexões TCP	Uma thread virtual por conexão para ler linhas	Simplifica o código ("uma thread por conexão") sem o custo de uma thread de plataforma por cliente
Conexões WebSocket	Os callbacks da biblioteca só enfileiram (fila de 256 itens por conexão), e uma thread virtual por conexão consome a fila em ordem	O processamento pode ser demorado (hash de senha, leitura do histórico) e os callbacks rodam em poucas threads compartilhadas. Estouro da fila encerra só aquela conexão (`fila_cheia`)
Escrita por sessão	Fila de saída de 1000 itens e thread escritora própria. O distribuidor só enfileira	Um cliente lento não bloqueia a distribuição aos demais. Fila estourada encerra só aquela sessão
Escrita no WebSocket	A escritora espera a biblioteca esvaziar o que ainda não saiu para a rede antes de enviar o próximo quadro	A biblioteca guarda sem limite o que não saiu. Esperar faz a fila da sessão encher e ativar o `fila_cheia`, em vez de acumular memória
Ordem das mensagens	Um `ReentrantLock` global (`ordemLock`) cobre a atribuição de `mid` e `ts`, a gravação no repositório e no log de envios e o enfileiramento do evento	Todos veem a mesma ordem, e a ordem do arquivo coincide com a da distribuição

Tema	Decisão	Por quê
Conjunto de sessões	`CopyOnWriteArrayList`	Muitas leituras (cada difusão) e poucas escritas (login e logout)
Arquivos	Cada arquivo tem um `RepositorioArquivo` com `ReentrantLock` próprio. Cada registro é escrito de uma vez, em uma linha própria, com o arquivo aberto em modo de anexação a cada operação	Escritas concorrentes não intercalam nem corrompem registros
Limpeza	A reescrita toma o mesmo lock do arquivo, grava em `<arquivo>.tmp` com `force()` e troca com `ATOMIC_MOVE`	Anexações concorrentes esperam o lock, e uma queda no meio deixa o original íntegro
Banco de dados	Uma conexão JDBC por operação, somente `PreparedStatement`	Cadastro e login são raros, e isso evita gerenciar pool
Reporte de erros	Cache dos últimos 10.000 `evid`, sob `synchronized`	Reenvio por perda de confirmação não duplica o log de erros

O `ordemLock` é global, então os envios de todos os clientes passam por uma seção crítica única, que inclui duas gravações em arquivo. Para uma sala de bate-papo de porte didático isso é suficiente, e a escolha troca vazão por uma ordem total simples de garantir.

10.2 Reconexão nos clientes

Os três clientes aplicam a mesma política de reconexão. O atraso base para a tentativa `n = 0, 1, 2, ...` é `min(1 s * 2^n, 30 s)`, ou seja, 1, 2, 4, 8, 16, 30, 30 segundos e assim por diante, com jitter de mais ou menos 20% sobre esse valor. São feitas no máximo 10 tentativas consecutivas. Depois disso o cliente mostra o estado "desconectado" e oferece tentar de novo, o que zera o contador. O contador também volta a zero após uma conexão bem-sucedida (login confirmado).

Para voltar à sala, o cliente repete o `login` automaticamente. Por isso login e senha ficam apenas na memória do processo e são apagados no logout. Se o relogin receber `CREDENCIAIS_INVALIDAS`, o cliente para de tentar. O histórico enviado em seguida é incorporado de forma idempotente por `mid`, então a reconexão não duplica mensagens. O estado visível ao usuário segue a sequência: desconectado, conectando, conectado, autenticado, e reconectando (com tentativa, máximo e atraso) quando a conexão cai depois do login. Um `ping` a cada 30 s mantém a conexão ativa, e há limite de 15 s por resposta do Middleware e de 5 s para abrir o socket.

10.3 Falhas e como o sistema reage

Falha	Comportamento
Mensagem fora do protocolo (JSON inválido, sem `op` ou `v`, `v` diferente de 1, operação desconhecida, grande demais)	Resposta `ok:false` com código fixo, e a conexão continua disponível
Operação que exige login em conexão sem sessão	`NAO_AUTENTICADO`
Campo cifrado que não decifra (chave errada, MAC inválido, Base64 inválido)	`DECIFRAGEM_FALHOU`. O log de erros recebe um registro de texto fixo, sem o conteúdo recebido. Nada é gravado no arquivo de conversas e a sessão segue utilizável
Exceção inesperada em uma operação	`ERRO_INTERNO` ao cliente e registro no log de erros. A conexão continua. Se a escrita no próprio log de erros falhar, o erro vai para `stderr`, sem recursão
PostgreSQL inacessível	`SERVICO_INDISPONIVEL` em cadastro e login. O Middleware continua de pé
Falha ao gravar a mensagem no repositório	`ERRO_INTERNO`. Nada é distribuído nem vai ao log de envios
Cliente lento (fila de saída cheia)	Só a sessão dele é encerrada (`fila_cheia`)
Cliente sem tráfego por 90 s (TCP)	Conexão encerrada (`timeout`)
Cliente fecha ou cai	A sessão é finalizada uma única vez, removida do distribuidor, e o evento `desconexao` é gravado no log de acessos com o motivo. As demais sessões não são afetadas
Falha na limpeza em um arquivo	Fica no log de erros. A limpeza dos demais arquivos prossegue e a próxima execução tenta de novo
Queda do processo durante a limpeza	O arquivo original permanece íntegro. O `.tmp` é removido na partida seguinte
Falha no handler agendado da limpeza	O corpo agendado captura todas as exceções, porque uma exceção não tratada cancelaria o agendamento em silêncio

---

11. Instalação e execução

Esta seção resume os passos para executar o sistema. O passo a passo completo, com execução em máquinas distintas, firewall e solução de problemas, está no `README.md` do repositório. Os comandos são executados a partir da raiz do repositório.

11.1 Pré-requisitos e banco de dados

O servidor precisa de JDK 21, Maven 3, PostgreSQL 16 e Python 3.11 ou superior (usado só para gerar as chaves). O Cliente_Desktop precisa de JDK 21 e Maven. O Cliente_Terceiro precisa de Python 3.11 ou superior com Tkinter e não tem outras dependências. O Cliente_Web precisa apenas de um navegador moderno.

Com o PostgreSQL em execução, é preciso criar o usuário, o banco e o esquema. `<SENHA_DB>` é uma senha escolhida por quem instala:

```
psql -d postgres -c "CREATE USER comunicador WITH PASSWORD '<SENHA_DB>';"
psql -d postgres -c "CREATE DATABASE comunicador OWNER comunicador;"
psql -h localhost -U comunicador -d comunicador -f sql/01_schema.sql
```

11.2 Chaves e configuração

As chaves de trânsito e do repositório são geradas por um script e gravadas em um arquivo que não é versionado. Depois, copia-se o arquivo de exemplo de configuração e ajustam-se `db.senha` e `ws.origens_permitidas`. Essa segunda propriedade deve conter a origem de onde a página do Cliente_Web é aberta, por exemplo `http://<IP_SERVIDOR>:8080`.

```
python3 scripts/gerar_chave.py --par > config/chaves.properties
chmod 600 config/chaves.properties
cp config/servidor.properties.example config/servidor.properties
```

11.3 Middleware

O Middleware é compilado com o Maven e iniciado pelo jar gerado. Ao subir, ele imprime uma linha com as portas em uso (TCP 9000, WebSocket 9001 e HTTP 8080). Configuração inválida encerra o processo com código 2, e porta ocupada com código 1.

```
mvn -q -DskipTests package
java -jar middleware/target/middleware-1.0.0-SNAPSHOT.jar config/servidor.properties config/chaves.properties
```

11.4 Clientes

Os clientes Desktop e Python leem a chave de trânsito do arquivo `.comunicador/chave_transito.txt`, na raiz do projeto. O arquivo contém só os 64 caracteres hexadecimais da propriedade `chave.transito`, entregues por um meio fora do socket. O Cliente_Desktop é compilado junto com o projeto e executado com o classpath do jar e das dependências copiadas para `cliente-desktop/target/lib`. O Cliente_Terceiro é executado diretamente pelo Python. Para o Cliente_Web, basta abrir no navegador a página servida pelo Middleware, informando a chave na tela de login ou no fragmento `#chave=<hex>` da URL.

```
# Cliente_Desktop (depois de mvn package e de copiar as dependências para target/lib)
java -cp "cliente-desktop/target/cliente-desktop-1.0.0-SNAPSHOT.jar:cliente-desktop/target/lib/*" \
  br.cefet.sd.cliente.desktop.ClienteDesktop --host=<IP_SERVIDOR> --porta=9000

# Cliente_Terceiro
cd cliente-python && python3 -m comunicador --host <IP_SERVIDOR> --porta 9000

# Cliente_Web: abrir no navegador
# http://<IP_SERVIDOR>:8080/#chave=<CHAVE_TRANSITO>
```

---

12. Demonstração

A demonstração não usa dados pré-carregados. As contas, as mensagens e os registros antigos são criados durante o próprio roteiro, que segue a ordem do enunciado.

Primeiro, cria-se uma conta pela tela de cadastro e mostra-se a linha correspondente na tabela `contas`, em que a senha aparece apenas como hash. Em seguida, tentam-se um cadastro com login repetido e um login com senha incorreta, que devem ser recusados, e depois o login correto. Os eventos `login_recusado` e `login_ok` aparecem no log de acessos. Com o Desktop, a Web e o Python conectados em contas diferentes, os três clientes trocam mensagens nos dois sentidos, usando o botão Enviar, a tecla Enter, um emoticon, o seletor de emojis, uma figurinha e um texto com acentos.

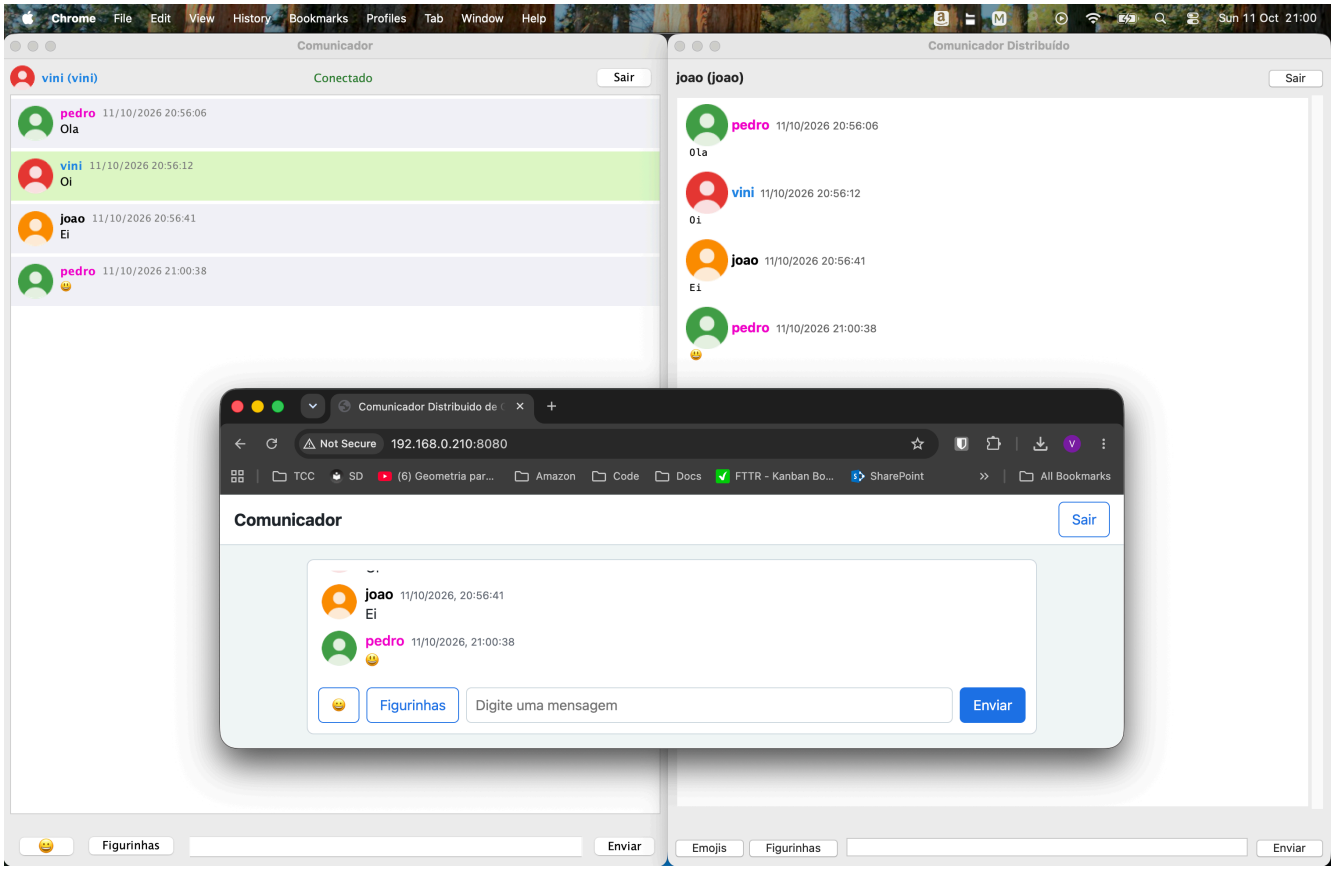


Figura 10: três clientes conectados simultaneamente e trocando mensagens.

Para mostrar a proteção dos dados, captura-se o tráfego do TCP e do WebSocket (comandos na seção 9.14) e abrem-se `dados/conversas.txt` e o `historico.txt` de um cliente. Em todos os casos, o `conteudo` aparece como Base64 e nenhum texto legível é visível. Depois, a decifragem do exemplo da seção 9.8 recupera `Ola, grupo!`.

A limpeza é demonstrada com o Middleware parado. O script `scripts/gerar_registros_antigos.py` acrescenta aos quatro arquivos registros vencidos e recentes, e o Middleware é iniciado em seguida. Como a limpeza roda antes da abertura das portas, basta comparar o número de linhas e os `ts` mais antigos antes e depois da partida.

```
python3 scripts/gerar_registros_antigos.py
wc -l dados/conversas.txt dados/logs/*.txt      # antes
java -jar middleware/target/middleware-1.0.0-SNAPSHOT.jar config/servidor.properties config/chaves.properties
wc -l dados/conversas.txt dados/logs/*.txt      # depois
```

Por fim, provocam-se duas falhas controladas. No cliente, com o Middleware parado, uma exceção não tratada no console do navegador fica no buffer offline e é enviada quando a conexão volta. No servidor, um cliente com chave de trânsito diferente da do servidor faz o Middleware responder `DECIFRAGEM_FALHOU` e registrar o erro. Os dois eventos aparecem em `dados/logs/erros.txt`, sem senhas nem conteúdo de conversa.

---

13. Conclusão, limitações e referências

13.1 Conclusão

O sistema entregue implementa um comunicador de grupo com um middleware central em Java e três clientes interoperáveis (Desktop em Swing, Web em HTML, CSS e JavaScript e Python em Tkinter) que conversam na mesma sala. As contas ficam em um banco cliente-servidor (PostgreSQL). O protocolo é em JSON, independente de linguagem, sobre TCP e WebSocket, com um único processador no servidor. As conversas e os históricos locais são cifrados com uma cifra manual implementada em três linguagens. O servidor mantém cinco repositórios com retenções próprias e limpeza na partida e periódica, e três logs separados recebem os erros do servidor e dos clientes, os acessos e os horários de envio.

As principais decisões foram usar WebSocket direto ao middleware para o navegador, atribuir `mid` e `ts` no servidor sob um lock de ordem, desacoplar a escrita de cada sessão por filas, tratar a incorporação de histórico por `mid` de forma idempotente e separar o tratamento de senhas da cifra das conversas. As limitações estão na seção 13.2.

13.2 Limitações

A cifra manual e o tratamento de senhas têm finalidade didática. A cifra foi escrita à mão, com estados e constantes escolhidos para o projeto, e não passou por análise criptográfica formal. Para uso real, seria preciso adotar uma cifra autenticada padronizada. O SHA-256 iterado usado nas senhas é mais fraco que bcrypt, scrypt ou Argon2, que não existem na biblioteca padrão do Java.

A proteção também não é de ponta a ponta. O Middleware decifra cada mensagem em memória e a cifra de novo para o canal seguinte, e a chave de trânsito é distribuída manualmente e compartilhada por todos os clientes. Como não há TLS, quem a conhece consegue ler o tráfego da sala. Os metadados necessários ao funcionamento (`mid`, `ts`, `tipo` e o perfil do remetente) ficam em claro. Por fim, o `ordemLock` é global, e os envios de todos os clientes passam por uma única seção crítica. Isso basta para uma sala de porte didático, mas limita a vazão.

13.3 Código-fonte

O código-fonte e os scripts de configuração estão disponíveis em https://github.com/tinicius/trabalho-1-sd. Os passos resumidos de instalação e execução estão na seção 11, e o passo a passo completo, com execução em máquinas distintas, está no `README.md` do repositório.

13.4 Referências

- Enunciado do Trabalho 1 de Sistemas Distribuídos (CEFET).
- RFC 6455, The WebSocket Protocol: https://www.rfc-editor.org/rfc/rfc6455
- Documentação do PostgreSQL 16: https://www.postgresql.org/docs/16/
- Documentação do Java SE 21: https://docs.oracle.com/en/java/javase/21/
- MDN, API IndexedDB: https://developer.mozilla.org/docs/Web/API/IndexedDB_API
- Documentação do módulo `tkinter` do Python: https://docs.python.org/3/library/tkinter.html
- Documentação do Mermaid, usado nos diagramas: https://mermaid.js.org/
- Visual Studio Code, IDE utilizada no desenvolvimento: https://code.visualstudio.com/